

Ch.3 Docker 다루기

머칠이 흘렀습니다. 컨테이너 한 대를 띄우고 내리는 일은 이제 손이 먼저 움직였습니다. 자투리 시간마다 docker 명령어를 두드려 본 덕에, 어제까지만 해도 낯설던 단어가 키보드 위로 자연스럽게 흘러나왔습니다. 노트 한 귀퉁이에는 마지막 줄로 적어 둔 한 문장이 그대로 남아 있었습니다.

'그런데 실제 서비스는 컨테이너 한 대로 안 끝날 텐데.'

화요일 오전 열 시. 회의실에 사람이 모였습니다. 팀장이 화이트보드 앞에서 새 프로젝트의 윤곽을 잡기 시작했습니다. 사내에서 쓸 작은 통합 사이트 하나를 새로 만든다는 이야기였습니다. 화면을 그릴 프론트엔드, 그 뒤를 받쳐 줄 백엔드, 회원 정보를 보관할 DB가 함께 도는 구조였습니다.

팀장이 펜을 내려놓고 오픈이 쪽을 봤습니다.

팀장 : "환경 구성은 Docker로 가요. 요즘 공부했잖아요."



그림 3-1. 회의 끝난 회의실에 남은 통합 사이트의 윤곽

오픈이는 고개를 끄덕였습니다. 회의가 끝난 뒤 자리로 돌아오는 길은 막막했습니다. 컨테이너 하나를 띄우는 건 익혔지만, 셋을 한꺼번에 띄우는 일은 처음이었습니다. 프론트엔드에서 백엔드로, 백엔드에서 DB로 요청이 흘러야 했고, 셋이 동시에 살아 있어야 했습니다.

'한 대씩은 띄워봤는데, 셋이 동시에 맞물려 도는 건 또 다른 이야기인데.'

자리에 앉아 모니터를 노려봤습니다. 사무실 형광등이 키보드 위로 길게 떨어졌고, 옆자리에서 마우스 클릭 소리만 또각또각 울렸습니다. 무엇보다 손대야 할지 그림이 잘 잡히지 않았습니다. 일단 퇴근 후에 노트북을 열기로 했습니다. 회의실에서 받은 통합 사이트, 그 셋을 한 대씩 컨테이너로 직접 부딪쳐 보면서 무엇이

막히는지부터 보기로 했습니다.

3.1 Dockerfile - 환경을 자동으로 만들기

3.1.1 프로비저닝

저녁 여덟 시. 식탁에 노트북을 펼쳤습니다. 가장 먼저 떠오른 장면은 지난주에 부딪혔던 수동 세팅이었습니다. 컨테이너에 들어가서 패키지 목록을 갱신하고, 편집기를 깔고, 파일을 만들고, 마지막에 commit으로 이미지를 남기는 그 과정이었습니다. 처음 한 번은 신기했습니다. 그런데 지금은 똑같은 일을 세 번 반복해야 했습니다. 프론트엔드용 한 번, 백엔드용 한 번, DB용 한 번.

'프로젝트 세팅할 때마다 매번 이 과정을 다시 해야 할까.'

명령어 한 줄을 잘못 치면 처음부터 다시 가야 했습니다. 패키지 이름을 한 글자만 빠뜨려도 그랬습니다. 손가락 하나로 두 시간을 날린 적도 있었습니다. 그러다 머릿속에 다른 그림이 들어왔습니다. 며칠 전 편의점에서 본 밀키트였습니다. 비닐을 뜯으면 손질된 재료와 양념이 한 봉지에 들어 있어서, 봉지를 열고 끓이기만 하면 끝나는 그 상자였습니다.

'도커도 저런 식으로 미리 준비해 둘 수 없을까. 누군가 한 번만 적어 두면, 다음에는 그 종이만 들고 있어도 같은 환경이 만들어지는 식으로.'

이런 방식이 도커에 이미 마련되어 있었습니다. 환경 구성을 정의 파일 하나에 적어 두고 자동으로 만들어 내는 작업을 **프로비저닝(Provisioning)** 이라고 부릅니다. 밀키트의 그 봉지처럼, 한 번만 잘 적어 두면 그 다음부터는 봉지를 뜯듯 환경이 차려집니다.

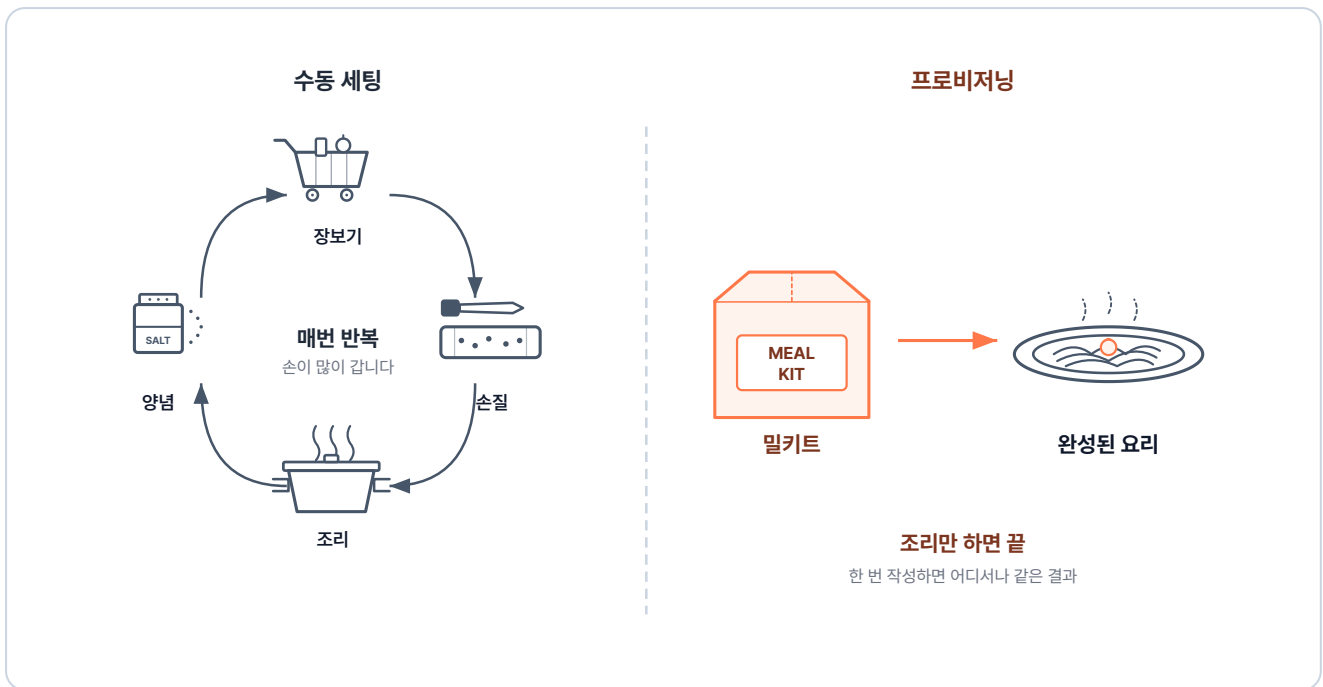


그림 3-2. 수동 세팅과 프로비저닝 비교

프로비저닝(Provisioning): 서비스가 동작할 수 있도록 환경(운영체제·패키지·설정 등)을 갖춰 사용할 가능한 상태로 만들어 두는 작업입니다.

도커에서 이 프로비저닝을 담당하는 정의 파일이 바로 **Dockerfile**입니다. 요리 레시피처럼 무엇을 어떤 순서로 준비할지 정확히 적어두면, 도커가 그대로 따라 자동으로 이미지를 만들어 줍니다.

Dockerfile: 컨테이너가 실행될 때 필요한 환경을 자동으로 구성해 주는 이미지를 만들기 위한 스크립트입니다. 베이스 이미지, 설치할 패키지, 복사할 파일, 실행할 명령을 순서대로 적어둡니다.

3.1.2 Dockerfile에서 컨테이너까지의 세 단계

레시피를 적는 종이가 있다고 끝이 아니었습니다. 그 종이를 들고 누가 어떻게 움직여서 식탁에 음식이 올라가는지가 이어집니다. Dockerfile에서 실제 컨테이너가 돌아가기까지는 크게 세 단계를 거칩니다.

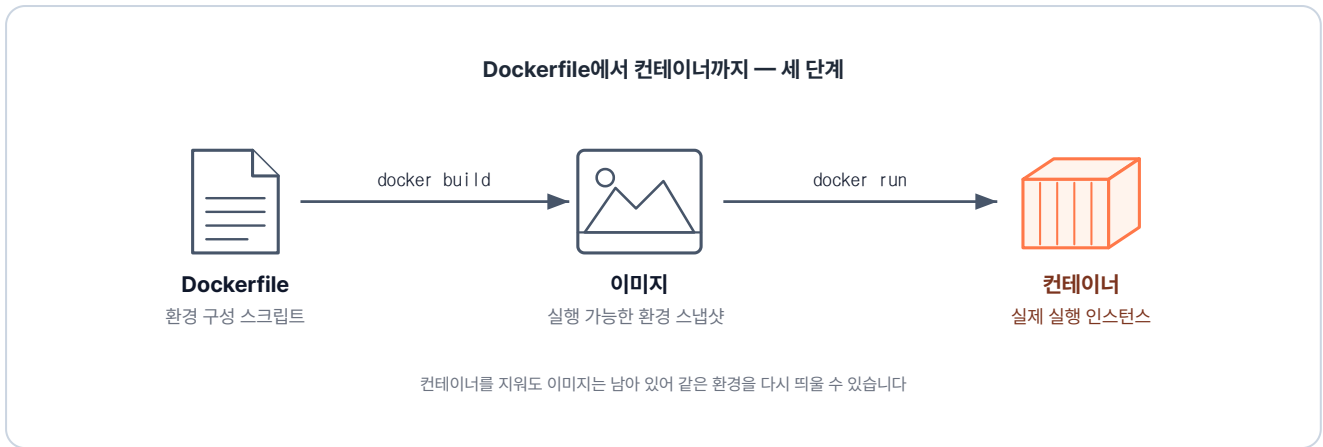


그림 3-3. Dockerfile → 이미지 → 컨테이너의 세 단계

1. **Dockerfile 작성:** 구축하고 싶은 환경을 텍스트 파일에 차례대로 적습니다.
2. **docker build:** 도커 엔진이 Dockerfile의 내용을 위에서 아래로 읽으며 실행합니다. 이 과정이 끝나면 결과물이 이미지로 저장됩니다.
3. **docker run:** 생성된 이미지를 기반으로 실제 컨테이너를 실행합니다.

컨테이너를 삭제하더라도 이미지는 그대로 남아 있습니다. 같은 환경이 필요할 때마다 언제든지 다시 띄울 수 있습니다. 지난주에 commit 명령어를 일일이 쳐서 만들었던 그 자리를 이제 Dockerfile이 채웁니다.

3.1.3 Dockerfile 기본 문법

레시피를 쓰려면 먼저 어떤 칸에 무엇을 적는지부터 알아야 했습니다. 오픈이는 Dockerfile에서 가장 자주 쓰이는 지시어를 표 한 장으로 정리했습니다. 레시피 칸을 채울 재료 같은 것이었습니다.

지시어	역할
FROM	베이스 이미지를 지정합니다. (어떤 환경에서 시작할지)
WORKDIR	컨테이너 내부에서 명령이 실행될 기본 디렉토리를 지정합니다.
COPY	호스트 컴퓨터의 파일을 컨테이너 안으로 복사합니다.
RUN	이미지를 빌드하는 동안 실행할 명령어입니다. (패키지 설치 등)
ENV	컨테이너 안에서 사용할 환경 변수를 설정합니다.
CMD	컨테이너 시작 시 실행할 기본 명령어입니다.
ENTRYPOINT	컨테이너 시작 시 반드시 실행되는 메인 프로세스입니다.

표가 손에 익자 첫 실습을 잡았습니다. Ubuntu 환경에 vim이 미리 깔려 있는 이미지를 만들어 보는 것이었습니다. 빈 폴더 하나를 만들고 그 안에 확장자 없이 `Dockerfile` 이라는 이름으로 파일을 하나 띄웠습니다.

```
# Dockerfile
FROM ubuntu:24.04           # 베이스 이미지
RUN apt update && apt install -y vim  # vim 패키지 설치
CMD ["/bin/bash"]           # 컨테이너 시작 시 bash 실행
```

저장한 뒤 터미널을 열었습니다. Dockerfile이 놓인 폴더에 들어가서 빌드 명령을 입력했습니다.

```
docker build -t ubuntu-vim .  # 현재 폴더의 Dockerfile로 이미지 빌드
```

실행 결과

```
$ docker build -t ubuntu-vim .
#7 naming to docker.io/library/ubuntu-vim:latest done
#7 unpacking to docker.io/library/ubuntu-vim:latest
#7 unpacking to docker.io/library/ubuntu-vim:latest 2.6s done
#7 DONE 9.2s
```

그림 3-4. docker build 실행 결과

빌드 로그가 한 줄씩 위로 올라갔습니다. 마지막 줄이 떨어지자 이미지가 완성되었습니다. 그 이미지로 컨테이너를 띄워 봤습니다. 들어가자마자 vim을 쳐 보니 별도의 설치 과정 없이 편집기가 그대로 뒀습니다.

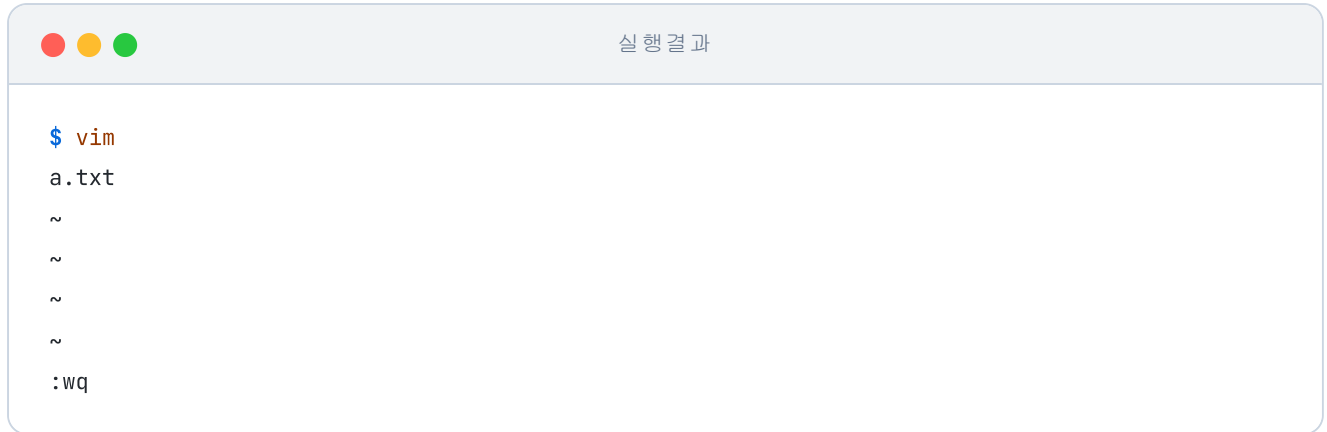


그림 3-5. vim이 이미 설치된 상태로 컨테이너 실행

지난주에는 같은 결과를 얻기까지 컨테이너 접속, 업데이트, 설치, 종료, 커밋까지 다섯 단계를 손으로 처리했습니다. 그 다섯 단계가 **Dockerfile 한 장과 명령어 한 줄**로 끝났습니다.

3.1.4 WORKDIR와 COPY

Dockerfile로 환경은 갖췄지만, 그 안에 들어갈 소스 코드는 아직이었습니다. 로컬에서 작업한 코드를 컨테이너 안으로 옮기는 방법이 필요했습니다. 이때 쓰이는 지시어가 **WORKDIR**와 **COPY**입니다.

오픈이는 일단 작은 것부터 시도했습니다. 빈 `index.html` 파일 한 장을 Dockerfile과 같은 폴더에 만들어 두는 것이었습니다.



그림 3-6. 폴더 및 파일 구조

그리고 Dockerfile에 두 줄을 보냈습니다. 컨테이너 안의 작업 디렉토리를 `/app`으로 정하고, 로컬의 `index.html`을 그 안으로 옮겨 넣는 설정이었습니다.

```
FROM ubuntu:24.04          # 베이스 이미지
WORKDIR /app                # 작업 디렉토리 설정
COPY ./index.html ./index.html # 로컬의 index.html을 컨테이너의 /app으로 복사
RUN apt update && apt install -y vim # vim 패키지 설치
CMD ["/bin/bash"]          # 컨테이너 시작 시 bash 실행
```

이렇게 하면 컨테이너에 들어갔을 때 곧장 `/app` 안에 떨어지고, 그 자리에 이미 `index.html` 이 놓여 있게 됩니다. 다시 빌드하고 실행해 봤습니다.

```
docker build -t ubuntu-html . # 현재 폴더의 Dockerfile로 이미지 빌드
docker run -it ubuntu-html    # 컨테이너 실행
ls
```

실행 결과

```
$ docker run -it ubuntu-html
root@5497d6efff3c:/app# ls
index.html
```

그림 3-7. 실행 결과 확인

터미널 프롬프트가 처음부터 `/app` 을 가리키고 있었고, 그 안에 `index.html` 이 들어 있었습니다. 컨테이너로 들어가서 파일을 직접 옮길 일이 사라진 것입니다. `exit` 을 입력해 컨테이너를 닫았습니다.

3.1.5 CMD와 ENTRYPOINT

문법 표 아래쪽 두 줄이 비슷해 보였습니다. CMD는 "실행할 기본 명령어", ENTRYPOINT는 "반드시 실행되는 메인 프로세스". 둘 다 '실행'이라는 단어가 들어 있어서 차이가 한눈에 들어오지 않았습니다.

'기본 명령어와 반드시 실행되는 메인 프로세스. 어쨌든 둘 다 실행한다는 말로 들리는데, 무엇이 다른 걸까.'

두 지시어는 성격이 조금 다릅니다. 카페의 커피 머신 한 대를 떠올리면 쉽습니다. 머신이 하는 본질적인 일은 '커피를 내린다'입니다. 손님이 따로 주문하지 않으면 그냥 '아메리카노'가 나옵니다. 손님이 라떼를 주문하면 머신은 라떼를 내리고, 에스프레소를 주문하면 에스프레소를 내립니다. 메뉴는 바뀌어도 '커피를 내린다'는 동작 자체는 그대로입니다. **ENTRYPOINT**가 그 '커피를 내린다'에 해당하고, **CMD**가 따로 주문이 없을 때 기본으로 나가는 '아메리카노'에 해당합니다.

CMD와 ENTRYPOINT

- **CMD**: 컨테이너가 시작될 때 실행할 **기본 명령**입니다. docker run 명령어를 칠 때 뒤에 다른 명령을 입력하면 기존의 CMD는 무시됩니다.
- **ENTRYPOINT**: 컨테이너가 시작될 때 **반드시 실행되어야 하는 메인 프로세스**입니다. 외부 명령어로 쉽게 덮어쓸 수 없도록 고정됩니다.

비유로 정리는 됐지만 실제로 어떻게 떨어지는지가 궁금했습니다. 오픈이 Dockerfile에 ENTRYPOINT 한 줄을 추가해 봤습니다. echo로 짧은 메시지를 찍는 단순한 구성이었습니다.

```
FROM ubuntu:24.04           # 베이스 이미지
WORKDIR /app                 # 작업 디렉토리 설정
COPY ./index.html ./index.html # 로컬의 index.html을 컨테이너의 /app으로 복사
RUN apt update && apt install -y vim # vim 패키지 설치
CMD ["/bin/bash"]           # ubuntu의 기본 프로세스
ENTRYPOINT ["echo", "컨테이너 실행"] # 컨테이너 시작 시 echo 실행
```

이미지를 다시 빌드하고 컨테이너를 띄워 봤습니다.

```
docker build -t ubuntu-entry . # 현재 폴더의 Dockerfile로 이미지 빌드
docker run -it ubuntu-entry    # 컨테이너 실행
```

실행 결과

```
$ docker build -t ubuntu-entry .
#7 naming to docker.io/library/ubuntu-entry:latest done
#7 DONE 0.3s
$ docker run -it ubuntu-entry
컨테이너 실행 /bin/bash
```

그림 3-8. ENTRYPOINT 실행 결과

ENTRYPOINT의 echo가 먼저 찍히고, 그 뒤에 CMD에 적혀 있던 `/bin/bash`가 인자처럼 따라붙어 한 줄로 나왔습니다. **컨테이너 실행 /bin/bash**라는 문구를 화면에 남기고 컨테이너는 곧 종료됐습니다.

ENTRYPOINT와 CMD가 만나면 벌어지는 일

`docker run 이미지명` 처럼 인자 없이 실행하면 '커피를 내린다' + '아메리카노'가 합쳐져 아메리카노가 나옵니다. 반면 `docker run 이미지명 라떼` 처럼 인자를 직접 넘기면 기본값이던 CMD가 '라떼'로 교체되어 '커피를 내린다' + '라떼'가 실행됩니다.

실제 도커 동작도 같은 방식입니다. ENTRYPOINT와 CMD가 함께 사용되면, 도커는 고정된 명령어인 ENTRYPOINT 뒤에 CMD를 꼬리표처럼 이어 붙여 하나의 프로세스를 생성합니다.

'명령어를 한 줄씩 손으로 치던 자리를 Dockerfile 한 장이 차지했네. 이게 프로비저닝이구나.'

이미지를 자동으로 만드는 준비를 끝낸 오픈이는 다음으로 회의실에서 받은 통합 사이트의 첫 칸을 짚어 보기로 했습니다.

3.2 NGINX - 요청을 앞에서 받아 나눠주기

3.2.1 NGINX를 서버 앞에 두는 이유

회의실에서 받은 사이트 구조에는 한 자리가 비어 있었습니다. 사용자가 어디로 들어오는지였습니다. 컨테이너 셋이 각자 포트를 들고 있다고 해서, 그 포트 번호를 사용자에게 그대로 알려 줄 수는 없었습니다. 외부로 내부 주소가 그대로 흘러나가는 구조가 됩니다.

'입구 하나만 두고 그 뒤에서 나눠 줄 방법은 없을까.'

떠오른 도구는 NGINX였습니다. 지난주에 한 번 띄워 본 적이 있는데, 그때는 '웹 서버구나' 정도로 흘려 들었습니다. 다시 들여다보니 NGINX는 웹 서버이면서 동시에 요청을 중간에서 받아 뒤로 넘겨 주는 **리버스 프록시(Reverse Proxy)** 역할도 했습니다. 사용자의 요청을 NGINX가 대신 받고, 뒤쪽 서버의 응답을 다시 사용자에게 돌려주는 구조였습니다.

NGINX: 가볍고 빠른 오픈소스 웹 서버이자 리버스 프록시입니다. 정적 파일 응답뿐 아니라 요청을 뒤쪽 서버로 전달·분배하고, 자주 쓰는 응답을 캐싱하는 등 서비스 앞단의 트래픽 관리에 두루 사용됩니다.

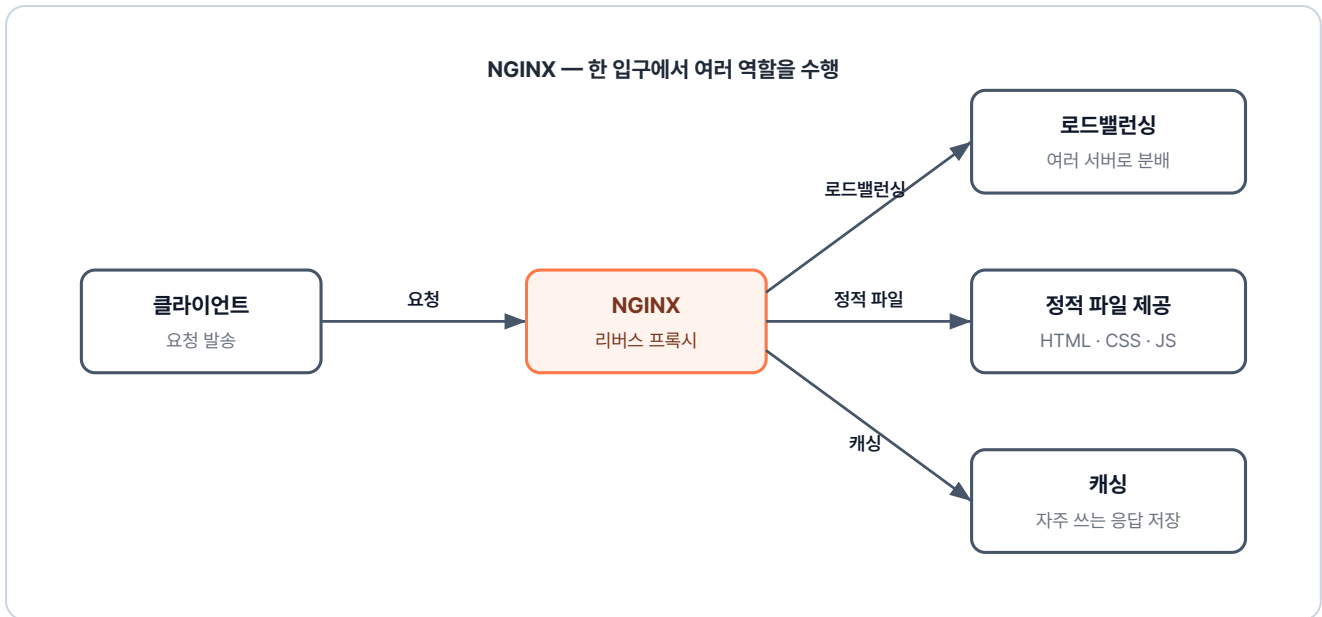


그림 3-9. NGINX가 앞에서 요청을 받아 뒤의 서버들로 전달

NGINX를 앞단에 세우면 실제 서버의 IP나 내부 포트를 사용자에게 노출할 일이 없습니다. 나중에 백엔드 서버를 두 대, 세 대로 늘려도 NGINX가 알아서 요청을 골고루 나눠 주는 로드밸런싱까지 받쳐 줍니다. 통합 사이트의 입구를 한 곳으로 모으고 싶었던 그 자리에 NGINX가 그대로 들어맞았습니다.

프록시 / 리버스 프록시 / 로드밸런싱

- **프록시**: 사용자가 인터넷상의 웹 사이트에 접속하려고 할 때, **사용자를 대신해 요청을 전달해 주는** 중간 통로입니다. 주로 보안을 위해 개인 정보를 감추거나, 자주 가는 사이트의 데이터를 미리 저장해 두어 접속 속도를 높일 때 사용합니다.
- **리버스 프록시**: 인터넷에서 **들어온 요청을 받아 내부 서버로 연결해** 줍니다. **NGINX의 주된 역할**이며, 실제 서버의 위치를 숨겨 안전하게 보호하고 관리하는 데 쓰입니다.
- **로드밸런싱**: 하나의 서버에 부하가 몰리지 않도록 요청을 여러 서버에 골고루 나눠주는 방식입니다.

3.2.2 NGINX 기본 문법 세 가지

NGINX의 동작은 전국 택배가 모이는 대형 분류 센터를 떠올리면 가깝습니다. 전국에서 들어온 택배가 한 곳에 쌓이고, 그 자리에서 주소지에 따라 지역 물류 센터로 갈라져 나갑니다. NGINX의 설정도 비슷한 흐름입니다.

- **upstream (서버 그룹 정의)**: 특정 지역을 담당할 **배송 센터(서버 그룹)** 를 묶어 이름을 붙이는 작업입니다. "**서울 센터**", "**부산 센터**" 처럼 물건을 넘겨줄 목적지를 미리 등록해 두는 것입니다.

- **location (요청 경로 매칭)** : 택배의 주소지(URL)를 확인하고 분류하는 게이트입니다. "주소가 '서울','부산' 등 무엇으로 시작하는가?" 를 확인하여 최종 행선지를 결정합니다.
- **proxy_pass (요청 전달)** : 분류된 택배를 지정된 물류 센터로 이동시키는 지시어입니다. "이 물품은 서울 센터로 보내" 라는 최종 명령입니다.

세 단어를 묶어서 뼈대 코드로 적으면 다음과 같습니다.

```
# nginx.conf
upstream backend {                                # backend라는 이름으로 서버 그룹 등록
    server host.docker.internal:8080;             # 그룹에 속한 실제 서버 주소
}

server {
    listen 80;                                     # 80번 포트로 들어오는 요청 대기

    location / {                                    # 모든 경로 요청
        proxy_pass http://backend;                # backend 그룹으로 넘김
    }
}
```

NGINX는 이런 설정을 `nginx.conf` 파일에 적어 둡니다. 옵션을 어떻게 묶느냐에 따라 같은 파일이 라우터가 되기도 하고, 로드밸런서가 되기도 합니다.

3.2.3 경로 기반 라우팅

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex01>

먼저 잡은 실습은 가장 단순한 것이었습니다. URL 경로에 따라 서로 다른 서버로 요청이 흘러가는 구조입니다. `/app1` 로 들어오면 1번 서버로, `/app2` 로 들어오면 2번 서버로 보내 주는 식이었습니다. 회의실에서 받은 사이트가 나중에 화면 한쪽은 게시판, 다른 한쪽은 회원 관리로 길이 갈라질 수 있다는 점에서 이 구조를 미리 손에 익혀 두면 좋겠다고 봤습니다.

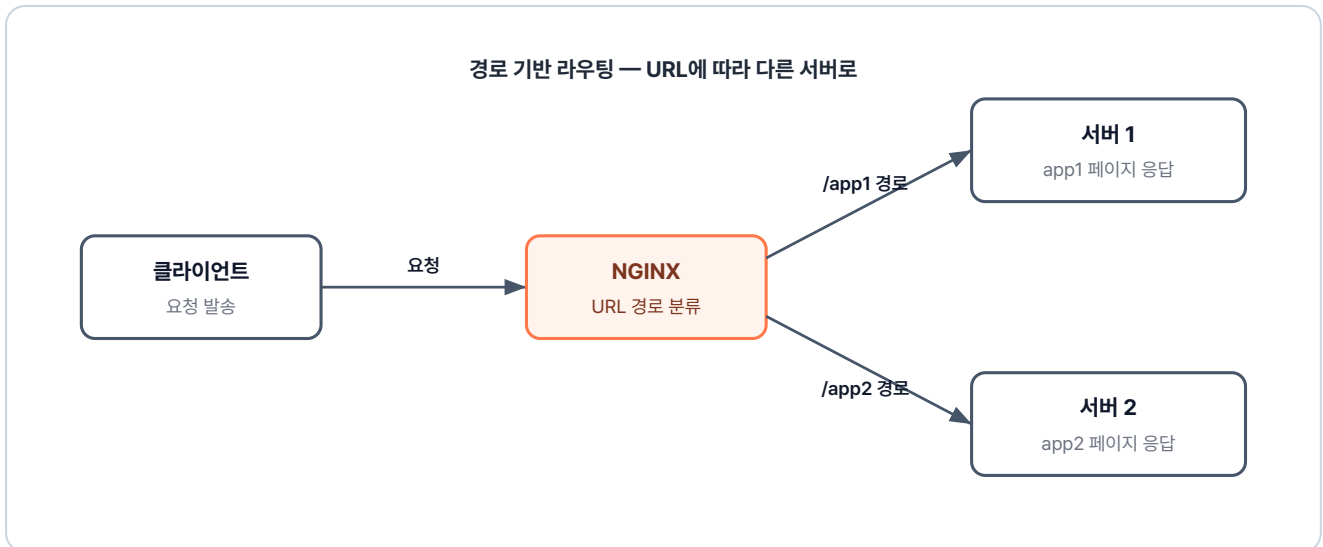


그림 3-10. 경로 기반 라우팅 구조

실습은 ex01 폴더에 들어 있었습니다. 폴더를 펼쳐 보니 컨테이너로 띄울 단위가 셋이었습니다. 화면을 들고 있는 app1과 app2, 그리고 둘 앞에 서서 길을 나누는 lb. 각자 자기 자리에 Dockerfile을 들고 있었습니다.

```

ex01/
├── app1/
│   ├── Dockerfile      # nginx 이미지 + index.html 복사
│   └── index.html       # app1 페이지
├── app2/
│   ├── Dockerfile      # nginx 이미지 + index.html 복사
│   └── index.html       # app2 페이지
├── lb/
│   ├── Dockerfile      # nginx 이미지 + nginx.conf 복사
│   ├── nginx.conf      # 로드밸런싱 + 경로 라우팅 설정
└── README.md           # 실습 안내
  
```

핵심은 lb 폴더의 `nginx.conf` 한 장이었습니다. 경로별로 어느 서버 그룹으로 흘러 보낼지 적는 자리였습니다.

ex01/lb/nginx.conf

```

upstream app1 {                                # app1 그룹 지정
    server host.docker.internal:8000;          # 호스트의 8000번 포트 = app1 컨테이너
}

upstream app2 {                                # app2 그룹 지정
    server host.docker.internal:9000;          # 호스트의 9000번 포트 = app2 컨테이너
}

server {
    listen 80;

    location /app1 {                            # /app1 주소로 오는 요청 분류
        proxy_pass http://app1;               # upstream의 app1 그룹으로 전달
    }

    location /app2 {                            # /app2 주소로 오는 요청 분류
        proxy_pass http://app2;               # upstream의 app2 그룹으로 전달
    }
}

```

설정 파일을 들고 있는 컨테이너도 만들어야 했습니다. lb 폴더의 Dockerfile이 그 자리였습니다.

ex01/lb/Dockerfile

```

FROM nginx                                     # NGINX 공식 이미지 사용
COPY nginx.conf /etc/nginx/conf.d/default.conf # 작성한 설정 파일을 기본 경로에 복사
ENTRYPOINT ["nginx", "-g", "daemon off;"]      # NGINX를 포그라운드로 실행

```

공식 NGINX 이미지를 그대로 베이스로 쓰고, 그 위에 방금 짰 `nginx.conf` 만 덮어 씌우면 라우터 한 대가 떨어집니다. app1과 app2의 Dockerfile도 같은 패턴이었습니다. NGINX 이미지에 각자의 `index.html` 만 얹은 구성이었습니다.

세 폴더가 준비되자 차례대로 빌드하고 실행했습니다.

```

docker build -t app1 ex01/app1 && docker run -dit -p 8000:80 app1 # app1 이미지 빌드
+ 호스트 8000 → 컨테이너 80
docker build -t app2 ex01/app2 && docker run -dit -p 9000:80 app2 # app2 이미지 빌드
+ 호스트 9000 → 컨테이너 80
docker build -t lb ex01/lb && docker run -dit -p 80:80 lb         # lb 이미지 빌드 +
호스트 80 → 컨테이너 80

```

브라우저 주소창에 `localhost:80/app1` 을 입력하자 1번 서버의 화면이 떴습니다. 주소를 `/app2` 로 바꿔서 다시 들어가니 이번에는 2번 서버 화면이 그 자리에 나타났습니다.



그림 3-11. /app1 경로로 접속한 결과

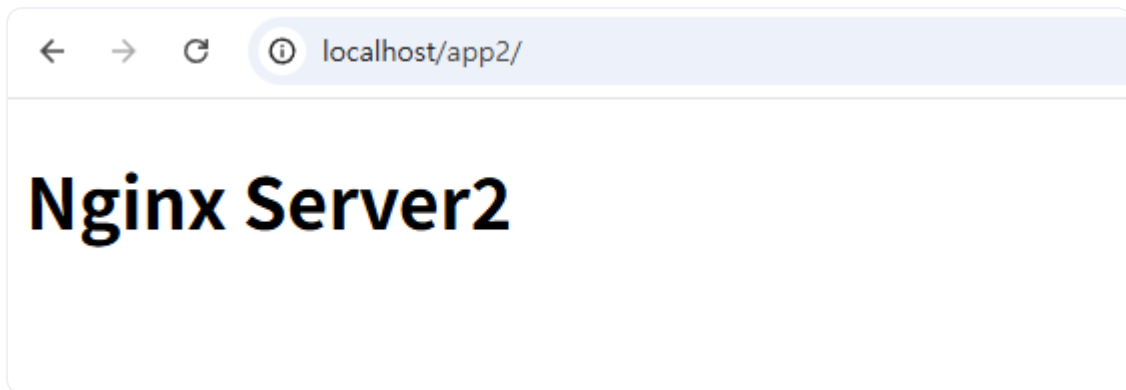


그림 3-12. /app2 경로로 접속한 결과

URL 끝의 한 글자가 달라졌을 뿐인데 다른 서버가 응답하고 있었습니다. `location` 이 요청을 가로채고, `proxy_pass` 가 정해진 upstream으로 흘려 보낸 결과였습니다.

3.2.4 컨테이너끼리 부르는 법

`host.docker.internal`이 왜 필요한가

오픈이는 `nginx.conf` 의 `host.docker.internal:8000` 한 줄이 눈에 들어왔습니다. lb 컨테이너에서 app1 컨테이너를 부르려는데 호스트 PC를 한 번 거쳐 가야 했습니다.

host.docker.internal: 컨테이너 내부에서 **호스트 PC**를 가리키는 특수 주소입니다. 컨테이너 안에서 `localhost`라고 입력하면 호스트 PC가 아닌 **컨테이너 자기 자신**을 가리키게 됩니다. 따라서 호스트 PC에 열려 있는 포트에 접근하려면 이 **별칭**을 사용해야 합니다.

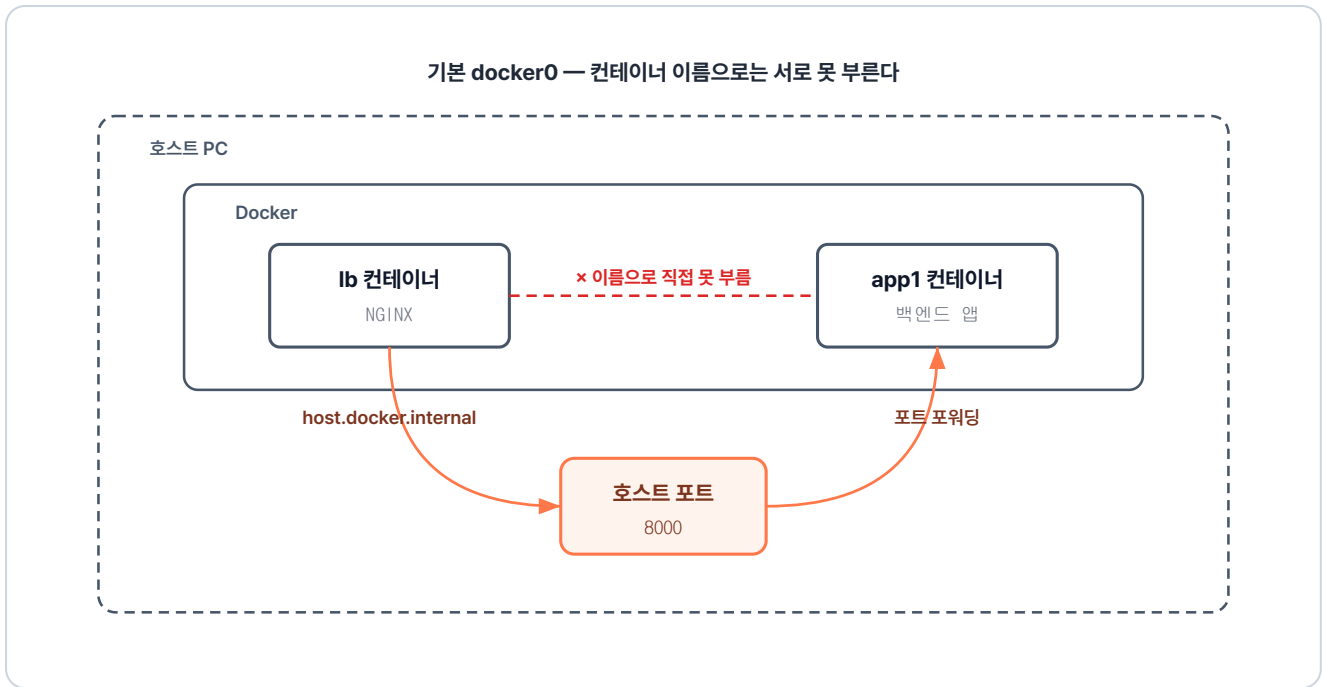


그림 3-13. lb 컨테이너 → 호스트 PC → app1 컨테이너로 가는 우회 경로

'같은 도커 위에서 도는 컨테이너끼리인데, 왜 호스트를 한 번 거쳐서 가야 하지.'

`docker run` 으로 컨테이너를 하나씩 띄우면 도커는 그 컨테이너를 **기본 네트워크**에 자동으로 끼워 넣습니다. 이 기본 네트워크에는 두 가지 한계가 있어서 우회가 필요합니다.

호스트 네트워크를 통해야 하는 이유

- **변동되는 IP** : 컨테이너를 재시작할 때마다 IP가 새로 부여되어 `nginx.conf`에 고정값으로 적어둘 수 없습니다.
- **이름으로 통신 불가** : 기본 네트워크에서는 `app1` 같은 컨테이너 이름으로 컨테이너 간 통신을 할 수 없습니다.

이러니 lb에서 app1을 부르려면 위치가 늘 일정한 호스트 PC를 통해야 했습니다. 호스트 PC를 가리키는 별칭이 바로 `host.docker.internal` 이었습니다.

같은 호스트 위에 있는 두 컨테이너인데, 매번 호스트 PC를 한 번 거쳐 돌아오는 길을 쓰고 있었습니다.

'매번 호스트를 한 번 거치지 않으려면 어떻게 해야 할까.'

사용자 정의 네트워크 — 이름으로 부르기

이 우회를 풀어 주는 도구는 챕터 2에서 이미 만져 봤습니다. **사용자 정의 네트워크(User-defined Network)** 였습니다. 챕터 2의 그림 2-17에서 본 것처럼, 사용자 정의 네트워크에서는 도커 내부 DNS가 컨테이너 이름을 IP로 대신 변환해 줍니다. 그래서 `host.docker.internal` 같은 우회 없이 컨테이너 이름을 그대로 적어 둘 수 있습니다.

실습에 필요한 명령어는 셋이었습니다.

명령	역할
<code>docker network create <네트워크이름></code>	새 네트워크 생성
<code>docker run --name <컨테이너이름> --network <네트워크이름> <이미지></code>	컨테이너를 해당 네트워크에 참여시키며 실행
<code>docker network ls</code>	현재 생성된 네트워크 목록 확인

세 줄로 같은 네트워크에 컨테이너를 묶으면 그때부터는 이름만으로 서로를 부를 수 있습니다.

'네트워크 한 칸을 따로 만들어서 셋을 모아 두면, 옆집 문을 그대로 두드릴 수 있겠네.'

3.2.5 로드밸런싱

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex02>

다음 실습은 같은 NGINX의 다른 얼굴이었습니다. 같은 서비스를 여러 대에 올려 두고, 들어오는 요청을 차례대로 나눠 보내는 일이었습니다.

NGINX는 `upstream` 블록 안에 `server` 줄을 여러 개 적어 두면, 들어오는 요청을 등록된 순서대로 한 곳씩 돌아가며 보내 줍니다. 카드 딜러가 카드를 한 장씩 차례대로 나눠 주듯, 서버 한 곳에 한 번씩 돌아가는 방식이었습니다. 이 방식을 **라운드 로빈(Round Robin)** 이라고 부릅니다.



그림 3-14. 라운드 로빈 로드밸런싱 구조

이번 실습은 ex02 폴더에 들어 있었습니다. 앞 실습과 달라진 자리는 한 군데였습니다. `nginx.conf`의 `upstream` 안에 `server` 줄이 두 개 들어 있고, 그 두 줄 모두 컨테이너 이름으로 적혀 있다는 점이었습니다.

ex02/lb/nginx.conf

```

upstream app1 {
    server app1-1:80;    # 첫 번째 서버. 컨테이너 이름으로 호출
    server app1-2:80;    # 같은 그룹에 두 번째 서버 추가
}

server {
    listen 80;
    location /app1 {
        proxy_pass http://app1;    # 두 서버에 번갈아 분배. 라운드 로빈 방식
    }
}
  
```

같은 이미지로 컨테이너를 두 번 띄울 계획이었습니다. 사용자 정의 네트워크에 모두 묶어 두면 호스트 포트를 따로 노출하지 않아도 컨테이너 이름만으로 서로 부를 수 있습니다.

```
# 1. 사용자 정의 네트워크 생성 (lb·app1-1·app1-2가 모두 이 네트워크에 묶입니다)
docker network create ex02-network

# 2. app1 이미지 빌드
docker build -t app1 ex02/app1

# 3. 같은 이미지로 컨테이너 두 개 실행 (--name으로 다른 이름을 부여해 도커 DNS에 등록)
docker run -dit --name app1-1 --network ex02-network app1 # app1-1 컨테이너를 ex02-network에 연결
docker run -dit --name app1-2 --network ex02-network app1 # app1-2 컨테이너를 ex02-network에 연결

# 4. lb(NGINX) 빌드 + 실행 (-p로 외부에 80 포트만 노출, 내부 통신은 네트워크 이름으로)
docker build -t lb ex02/lb
docker run -dit --name lb --network ex02-network -p 80:80 lb
```

브라우저에서 `localhost:80/app1`에 들어가서 새로고침을 여러 번 했습니다. 화면에 찍힌 HTML은 매번 똑같았습니다.



그림 3-15. /app1 경로로 접속한 결과 (라운드 로빈)

화면만으로는 어느 서버가 응답한 건지 알 길이 없었습니다. 두 컨테이너가 같은 이미지였기 때문입니다. 두 서버 양쪽으로 진짜 요청이 갈라져 들어갔는지 확인하려면 각자의 로그를 직접 들여다봐야 했습니다.

`docker logs` 명령을 두 번 쳤습니다.



그림 3-16. app1-1 컨테이너 로그에 찍힌 요청



그림 3-17. app1-2 컨테이너 로그에 찍힌 요청

새로고침 한 번에 한쪽 로그가 한 줄씩 늘었습니다. 다시 새로고침을 누르면 이번에는 다른 쪽 로그에 한 줄이 떨어졌습니다. 두 컨테이너에 한 번씩 번갈아 요청이 닿고 있었습니다. `server` 줄 하나를 더 보냈을 뿐인데 NGINX가 트래픽을 양쪽으로 갈라 보내 주고 있었습니다. 별도 옵션 없이도 라운드 로빈이 기본으로 켜져 있는 덕분이었습니다.

'서버가 한 대 더 늘어도 upstream에 줄 하나 보태면 끝이라니, 생각보다 깔끔한데.'

3.2.6 캐싱

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex03>

로드밸런싱 로그를 보니 양쪽 컨테이너 모두 똑같은 HTML을 매번 새로 만들어 응답하고 있었습니다. 같은 한 줄짜리 페이지를 두 대가 번갈아 가며 다시 그리고 있는 모양이었습니다.

'단순한 HTML 한 장을 매번 백엔드까지 가서 만들어 주고 있다니. 사람이 늘면 이건 또 다른 부담이 되겠는데.'

이 비효율을 풀어 주는 방법이 있었습니다. 자주 입는 옷을 옷장 깊숙이 넣지 않고 옷걸이에 따로 걸어 두면 다음 날 그 옷걸이에서 바로 꺼낼 수 있습니다. NGINX 앞단에 자주 나가는 응답을 잠깐 걸어 두면, 다음에 같은 요청이 들어왔을 때 백엔드까지 가지 않고 NGINX가 그 자리에서 돌려 줄 수 있습니다. 이 동작을 **캐싱 (Caching)** 이라고 부릅니다.

캐싱이 켜진 응답에는 두 가지 상태가 따라붙습니다.

상태	의미
MISS	캐시에 저장된 응답이 없어 백엔드 서버까지 다녀온 상태
HIT	캐시에 보관된 응답을 백엔드 거치지 않고 바로 돌려준 상태



그림 3-18. 첫 번째 요청 (MISS) — 캐시가 비어있어 백엔드까지 요청 후 응답을 캐시에 저장

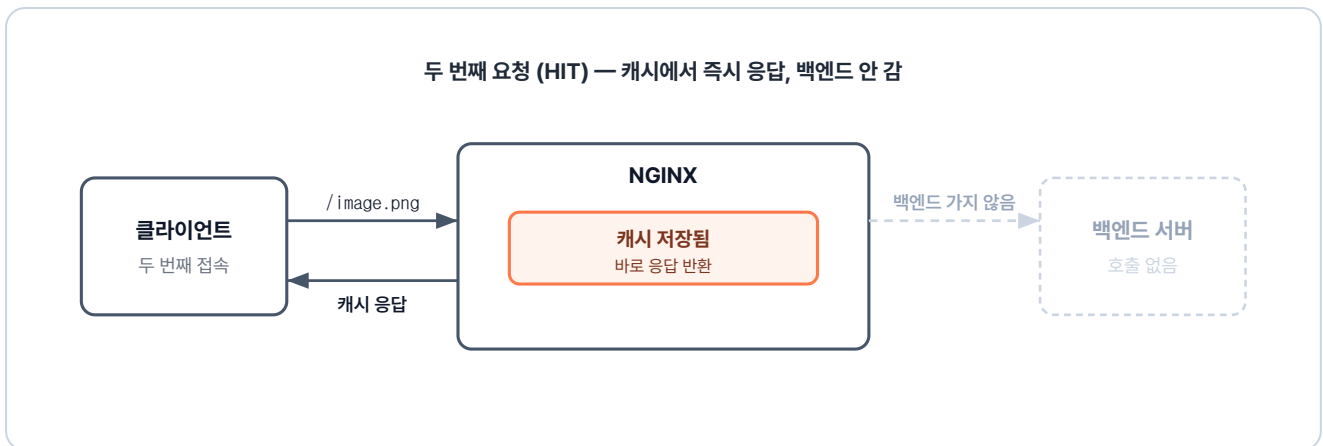


그림 3-19. 두 번째 요청 (HIT) — 캐시에 저장된 응답을 바로 반환, 백엔드 접근 없음

캐싱 실습은 ex03 폴더에 들어 있었습니다. 응답으로 이미지 파일 한 장을 내려주는 작은 파이썬 API 서버에 NGINX를 한 겹 씌운 구성이었습니다.

```

ex03/
├── api/
│   ├── app.py           # 백엔드. Flask 기반
│   ├── Dockerfile       # /image 라우트에서 image.png 반환
│   ├── image.png        # Python 이미지 + app.py·image.png 복사
│   └── image.png         # 응답으로 내려주는 이미지 파일
├── nginx/
│   ├── Dockerfile       # NGINX. 캐싱 + 프록시
│   ├── nginx.conf       # nginx 이미지 + nginx.conf 복사
│   └── nginx.conf        # proxy_cache 설정 + 프록시 라우팅
└── README.md            # 실습 안내
    
```

설정 파일에서 새로 등장한 지시어는 두 줄이었습니다. `proxy_cache_path` 는 캐시를 저장할 자리와 크기를 만들어 두는 칸이고, `proxy_cache` 는 그 칸을 어떤 location에서 결지를 정하는 스위치였습니다.

ex03/nginx/nginx.conf

```
# 캐시를 저장할 경로와 메모리 공간 이름을 선언합니다. (http{} 블록 내부에 위치)
proxy_cache_path /var/cache/nginx keys_zone=my_cache:10m;

server {
    listen 80;

    location / {
        proxy_pass http://api:5000;
        proxy_cache off; # 일반 경로는 캐시를 끕니다.
    }

    location = /image.png { # 이미지 파일 요청에 대해서만
        proxy_pass http://api:5000; # 선언한 캐시 공간을 사용합니다.
        proxy_cache my_cache; # 정상 응답을 1분 동안 보관
        proxy_cache_valid 200 1m; # HIT/MISS 표시
        add_header X-Cache-Status $upstream_cache_status always; # 백엔드 캐시 헤더 무시. NGINX 설정
        proxy_ignore_headers Cache-Control Expires; #
    }
}
```

우선 }

터미널을 열고 실습 환경을 띄우기 전에 정리부터 했습니다. ex02에서 띄워 둔 lb 컨테이너가 80 포트를 잡고 있어서, 이 자리를 비워 줘야 새 nginx-cache가 같은 포트를 들 수 있었습니다.

```
# 1. 사용자 정의 네트워크 생성
docker network create ex03-network

# 2. api(Flask) 이미지 빌드 + 실행
docker build -t api ex03/api
docker run -dit --name api --network ex03-network api

# 3. nginx 캐싱 빌드 + 실행 (-p로 외부에 80 포트만 노출)
docker build -t nginx-cache ex03/nginx
docker run -dit --name nginx-cache --network ex03-network -p 80:80 nginx-cache
```

브라우저에서 `localhost:80/image.png` 를 열었습니다. 화면에 이미지가 떴습니다.



그림 3-20. 캐싱 실습 — 이미지 응답

이미지가 떴다고 끝난 게 아니었습니다. 정말 두 번째 요청이 백엔드까지 가지 않고 NGINX 안에서 끝난 건지 확인해야 했습니다. **개발자 도구(F12, 브라우저 디버그 창)**의 Network 탭을 열고, 브라우저 자체 캐시가 끼어들지 못하도록 **Disable cache**를 체크한 뒤 **응답 헤더**(서버가 응답에 붙여 보내는 메타 정보)를 들여다봤습니다. 첫 요청에서는 **X-Cache-Status** 값이 예상대로 **MISS**로 찍혀 있었습니다.

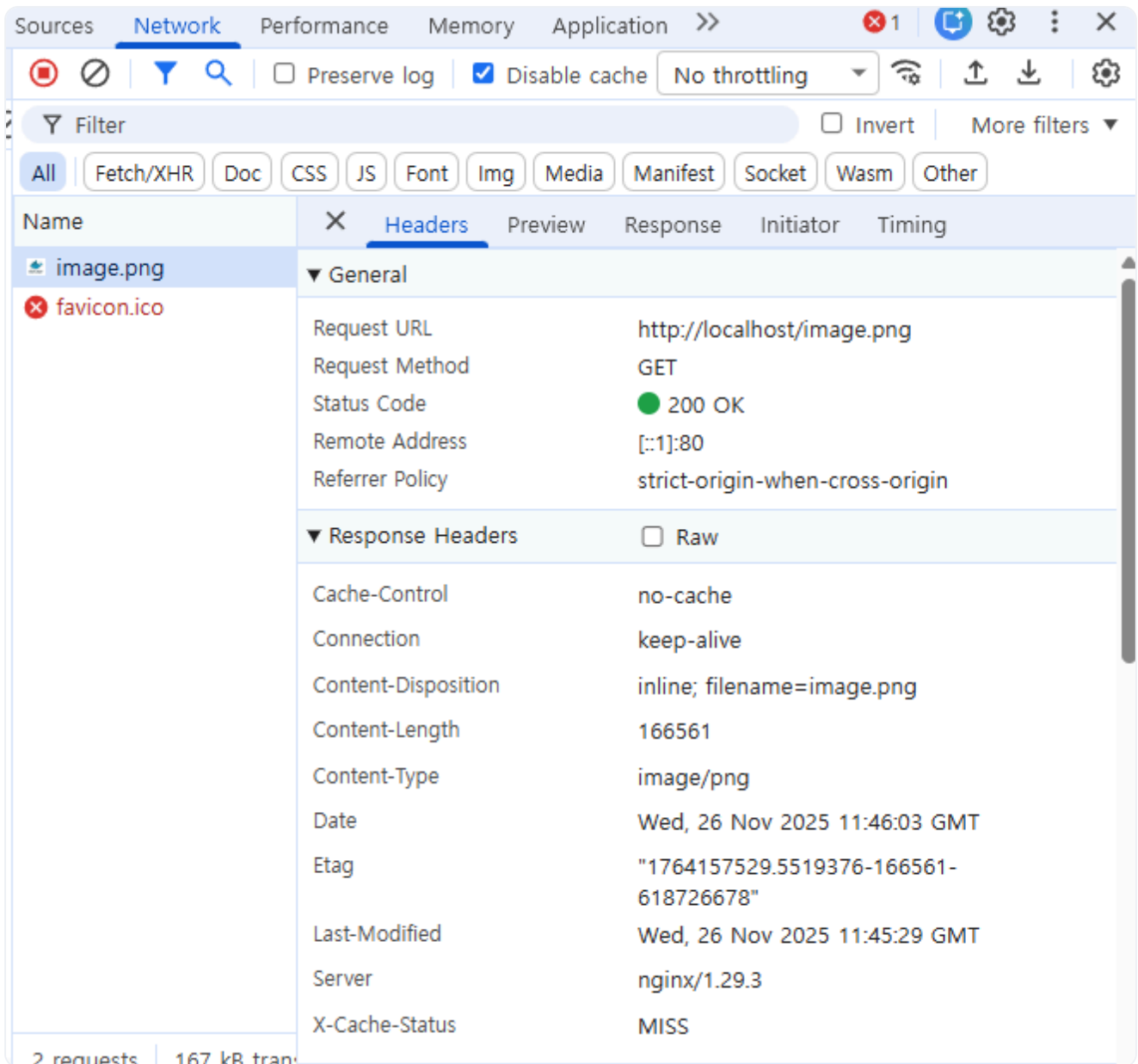


그림 3-21. X-Cache-Status: MISS 확인

새로고침을 한 번 더 눌렀습니다. 같은 자리에 찍히는 값이 이번에는 **HIT**로 바뀌어 있었습니다. 두 번째 요청은 백엔드 컨테이너까지 닿지 않았습니다. NGINX가 자기 자리에 저장해 둔 응답을 그대로 돌려 준 결과였습니다.

'어, 진짜 HIT이 떴네.'

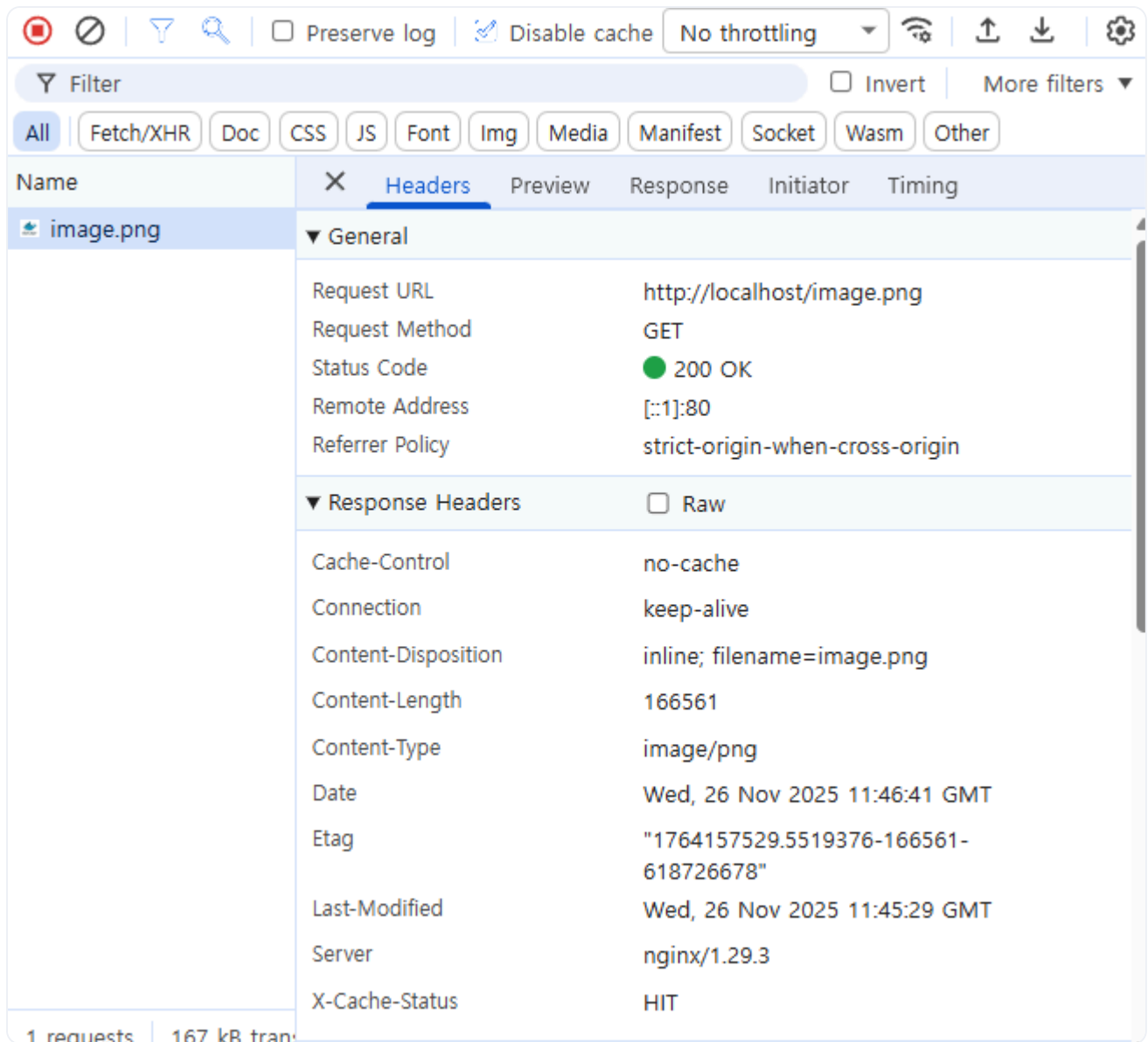


그림 3-22. X-Cache-Status: HIT 확인

세 가지 실습을 끝내자 `nginx.conf`의 패턴이 한눈에 들어왔습니다. 어떤 요청을(location) 어디로 보내고(proxy_pass) 그 사이에 어떤 옵션을 끼울지 정하는 흐름이 매번 똑같았습니다.

3.3 Redis - 서버 여러 대가 함께 쓰는 공용 저장소

3.3.1 서버 여러 대면 생기는 세션 문제

NGINX로 트래픽을 갈라 보내는 흐름은 손에 익었습니다. 그런데 다음 날 점심을 먹고 자리에 돌아왔을 때 옆자리 동료가 의자를 돌려 말을 걸어 왔습니다.

동료 : "오픈 씨, 어제 띄운 그거 한번 만져 봤는데요. 로그인은 잘 되는데 다음 페이지 누르면 다시 로그인하라고 떠요. 자꾸 튕겨 나가요."

오픈이는 의자를 끌어당기며 화면을 같이 들여다봤습니다. 동료가 로그인 버튼을 눌렀습니다. 환영 화면이 한 번 떴고, 메뉴를 한 번 누르자 로그인 페이지로 다시 떨어졌습니다. 한 번 더 눌러도 같은 자리로 떨어졌습니다.

'로그인은 됐는데 왜 다음 페이지에서 풀려 버리지.'

원인은 로그인 기록이 처음 들어간 서버의 메모리에만 들어 있었기 때문입니다. 사용자가 로그인을 하면 서버는 "이 사용자는 인증되었습니다"라는 정보를 자기 메모리에 잠시 적어 둡니다. 이런 기록을 **세션(Session)** 이라고 부릅니다.

세션(Session): 사용자가 로그인했을 때 서버가 생성하는 임시 기록입니다. 서버는 "이 사용자는 인증되었습니다"라는 정보를 자신의 메모리에 보관하고, 이후 요청이 올 때마다 이 기록을 대조해 로그인 상태를 유지합니다.

서버가 한 대일 때는 문제가 보이지 않습니다. 그런데 두 대 이상이 되면 메모리가 서버마다 따로 있어서, 1번 서버에 적어 둔 기록을 2번 서버는 알 길이 없습니다. 동료가 처음 로그인할 때 1번 서버가 받아 줬고, 다음 클릭은 라운드 로빈을 따라 2번 서버로 흘러갔습니다. 2번 서버는 이 사람이 누구인지 모르니 다시 로그인 화면을 보여 줄 수밖에 없었습니다.

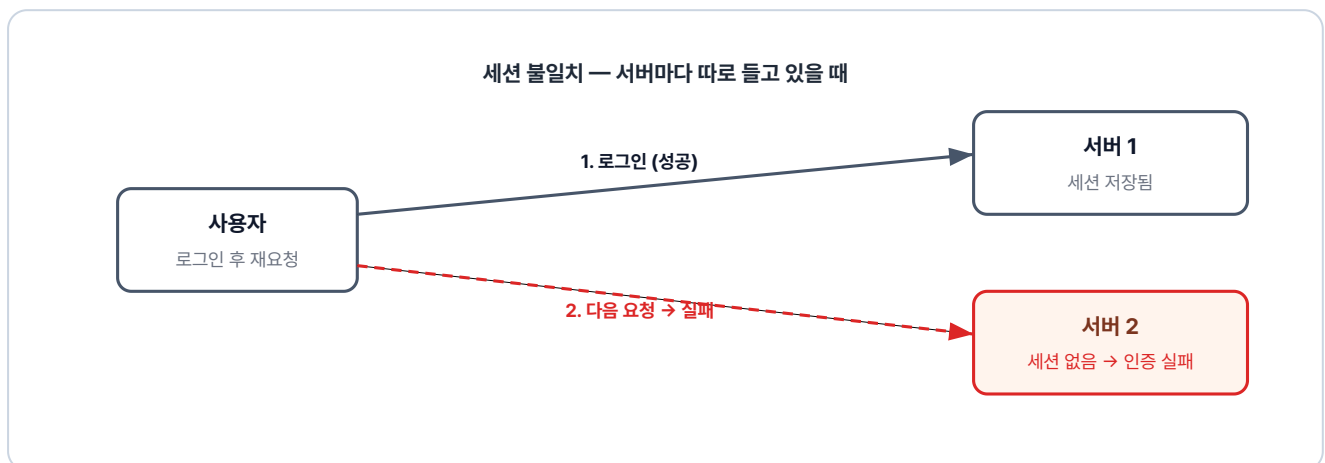


그림 3-23. 세션 불일치 — 1번 서버에 저장된 세션이 2번 서버엔 없어 인증 실패

3.3.2 Redis - 서버들의 공용 화이트보드

답은 단순했습니다. 세션을 각 서버의 메모리에 따로 두지 말고, 모든 서버가 같이 들여다볼 수 있는 한 자리에 두면 됐습니다.

회의실의 모습을 떠올리면 가까웠습니다. 사람마다 들고 있는 개인 노트에 적은 내용은 본인만 볼 수 있습니다. 반면 앞에 걸린 화이트보드에 적은 내용은 회의실에 있는 모두가 그 자리에서 같이 봅니다. 서버들이 들여다볼 화이트보드 역할을 해 주는 도구가 **Redis** 입니다.

Redis: 메모리 기반의 키-값(Key-Value) 데이터베이스입니다. 데이터를 디스크가 아닌 메모리에 저장하기 때문에 처리 속도가 매우 빠릅니다. 그래서 세션 저장소나 캐싱처럼 짧은 시간 안에 빈번한 읽기/쓰기가 필요한 곳에 주로 사용됩니다.

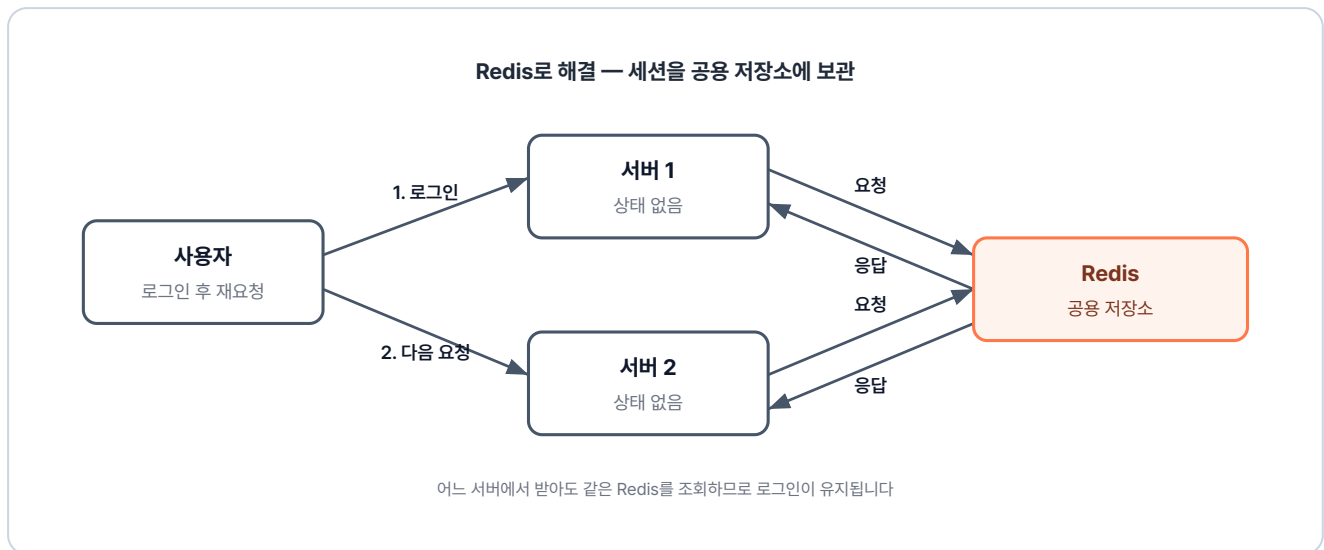


그림 3-24. Redis로 해결 — 세션을 공용 저장소에 보관하여 어느 서버에서든 조회 가능

1번 서버가 로그인을 처리한 다음 그 기록을 Redis에 적어 둡니다. 그 다음 요청이 라운드 로빈을 타고 2번 서버로 흘러가도, 2번 서버는 자기 메모리 대신 Redis 화이트보드를 들여다봅니다. 같은 사용자라는 사실이 그 자리에서 확인됩니다.

서버를 두 대로 늘려도, 세 대로 늘려도 사용자는 한 번 로그인하면 그 상태가 끊기지 않습니다.

3.3.3 실습: Redis로 세션 공유

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex04>

화면에 동료를 잠깐 두고 ex04 폴더를 열었습니다. Redis는 공식 이미지를 그대로 쓰면 되니 직접 만들 이미지는 작은 API 서버 한 대뿐이었습니다.

```
ex04/
├── api/
│   ├── Dockerfile      # Python 이미지 + app.py 복사
│   └── app.py           # Redis에 값을 저장/조회하는 간단한 API
└── README.md           # 실습 안내
```

`app.py` 는 두 개의 경로를 들고 있는 작은 서버입니다. `/save` 로 들어오면 값을 Redis에 적어 두고, `/read` 로 들어오면 그 값을 다시 꺼내 응답합니다.

이 `app.py` 를 띄울 컨테이너의 Dockerfile은 다음과 같습니다.

ex04/api/Dockerfile

```
FROM python:3.10-alpine      # 가벼운 파이썬 이미지 사용
WORKDIR /app                 # 작업 디렉토리 설정
COPY app.py .                # 위의 app.py 복사
RUN pip install flask redis  # Flask + Redis 클라이언트 설치
CMD ["python", "app.py"]     # 컨테이너 시작 시 app.py 실행
```

가벼운 파이썬 이미지 위에 flask와 redis 라이브러리만 얹어 `app.py` 를 띄우는 구성이었습니다. Redis 자체는 공식 이미지를 그대로 가져다 쓸 예정이라 직접 빌드할 자리는 이 api 한 곳뿐이었습니다.

준비가 끝나자 네트워크부터 만들고, 세 컨테이너를 같은 네트워크에 묶어 차례대로 띄웠습니다.

```
# 1. 사용자 정의 네트워크 생성
docker network create ex04-network

# 2. Redis 컨테이너 실행 (-p는 호스트에서 확인용으로 노출, 같은 네트워크 내 통신에는 불필요)
docker run -d --name redis --network ex04-network -p 6379:6379 redis

# 3. API 서버 두 대 실행 (같은 이미지, 다른 포트)
docker build -t api ex04/api
docker run -d --name api1 --network ex04-network -p 5001:5000 api
docker run -d --name api2 --network ex04-network -p 5002:5000 api
```

이제 데이터가 양쪽 서버 사이에서 같이 보이는지 확인할 차례였습니다. 먼저 api1 서버 (`localhost:5001/save`)에 접속해 값을 한 번 적어 두고, 그 다음에 api2 서버 (`localhost:5002/read`)로 들어가 같은 값을 꺼내 봤습니다.

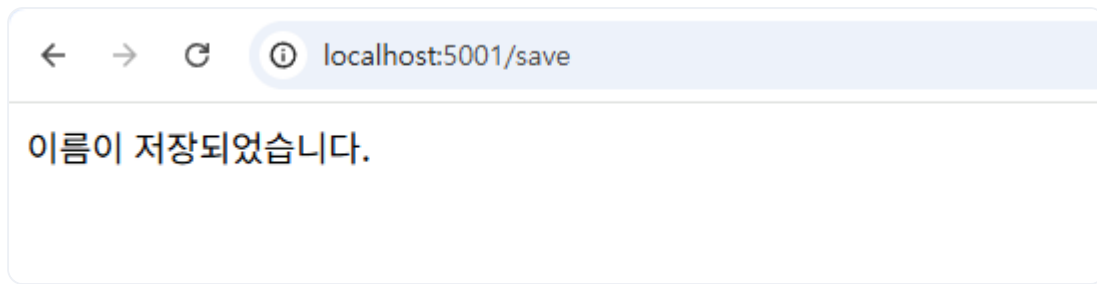


그림 3-25. api1에서 데이터 저장

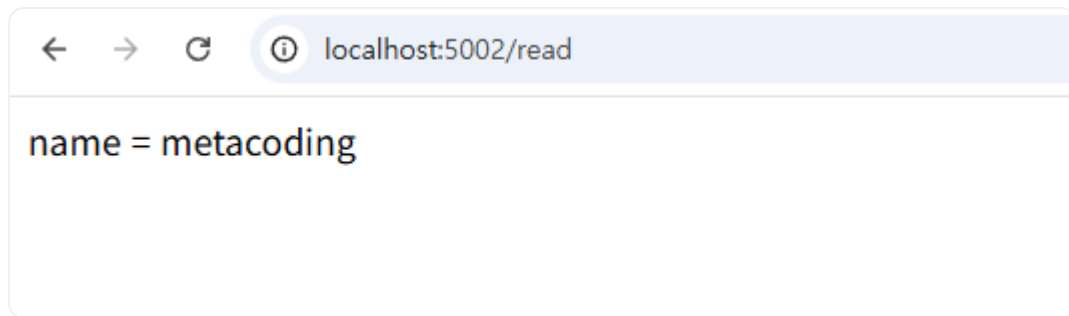


그림 3-26. api2에서 같은 데이터 조회

api1에 적어 둔 값이 api2 화면에 그대로 나왔습니다.

자리로 다시 돌아온 동료에게 화면을 돌려 봤습니다.

오픈이 : "이제 한 번 로그인하면 다음 페이지에서 안 풀려요. Redis라는 공용 저장소를 하나 뒀서 두 서버가 같은 자리를 보게 했어요."

동료 : "아, 그래서 다시 로그인 화면이 안 뜨는 거네요."

각 서버가 따로 들고 있던 기록이 한 자리에 모아져, 사용자 입장에서 매번 끊기던 흐름이 이어졌습니다.

3.4 MySQL - 영구 데이터의 자리

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex05>

Redis는 메모리 위에서 도는 도구라서, 컨테이너를 한 번 내렸다가 다시 띄우면 그 안의 값이 모두 사라집니다.

'세션처럼 잠깐 들고 있다 사라져도 되는 데이터는 괜찮은데, 회원 정보나 게시글 같은 건 어디에 두지. 컨테이너를 재시작해도 그대로 남아 있어야 할 텐데.'

오래 남아야 하는 데이터에는 따로 데이터베이스 서버가 필요했습니다. 회의실에서 받은 통합 사이트의 회원 정보를 둘 자리로 MySQL을 골랐고, 이 역시 컨테이너로 띄워 보기로 했습니다.

MySQL: 표 형태로 데이터를 보관하는 관계형 데이터베이스(RDBMS)입니다. 회원 정보·게시글처럼 영구적으로 남아야 하는 데이터를 저장할 때 주로 사용합니다. 도커 공식 이미지로 제공되어 컨테이너로 손쉽게 띄울 수 있습니다.

ex05/db/Dockerfile

```
FROM mysql                                # MySQL 공식 이미지 사용
COPY init.sql /docker-entrypoint-initdb.d  # 첫 기동 시 자동 실행될 SQL 복사
ENV MYSQL_USER=metacoding                 # 사용자 계정 설정
ENV MYSQL_PASSWORD=metacoding1234         # 사용자 비밀번호
ENV MYSQL_ROOT_PASSWORD=root1234          # root 비밀번호
ENV MYSQL_DATABASE=metadb                 # 기본 생성할 데이터베이스 이름
CMD ["--character-set-server=utf8mb4", "--collation-server=utf8mb4_unicode_ci"]
```

Dockerfile에서 눈여겨볼 자리는 두 군데입니다.

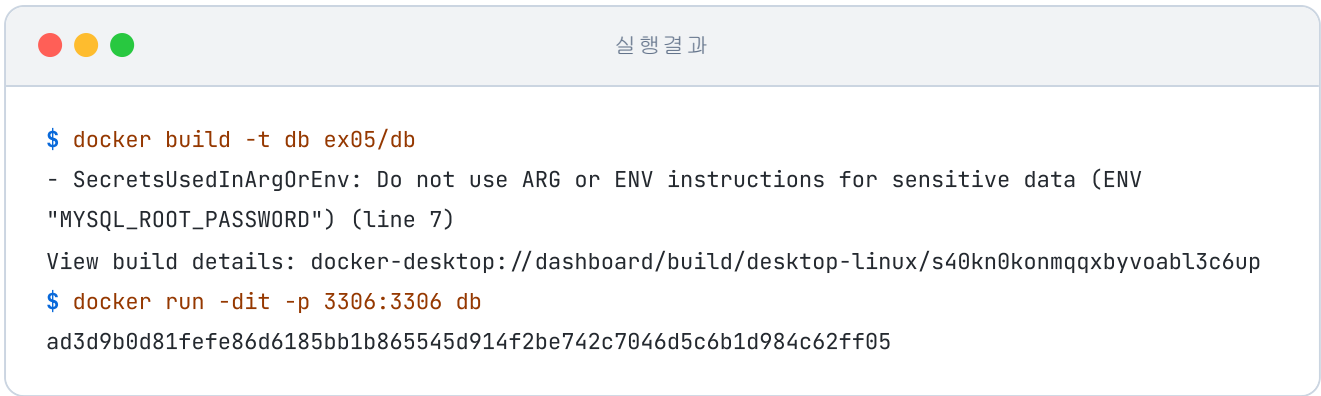
- **/docker-entrypoint-initdb.d** : MySQL 공식 이미지가 제공하는 특수 경로입니다. 여기에 init.sql 파일을 넣어두면 컨테이너가 처음 실행될 때 자동으로 SQL 문을 실행해 테이블과 초기 데이터를 만들어줍니다.
- **환경 변수(ENV)** : 네 개의 환경 변수를 통해 계정과 DB 이름을 미리 설정합니다. 실제 접속 시 이 설정값들을 사용하게 됩니다.

Dockerfile에 비밀번호를 직접 적는 방식

이번 실습에서는 과정을 단순하게 보여드리기 위해 비밀번호를 Dockerfile에 직접 적었습니다. 하지만 실무에서 이런 방식은 매우 위험합니다. 이미지를 빌드하는 순간 이미지를 가진 사람 누구나 비밀번호를 볼 수 있기 때문입니다. 그래서 실제 서비스를 운영할 때는 비밀번호 같은 민감한 정보를 이미지 내부에 기록하지 않고 외부에 따로 보관해 두었다가, 컨테이너가 실행되는 시점에만 살짝 건네주는 방식을 사용합니다. 이 방식은 쿠버네티스에서 사용해보겠습니다.

ex05 폴더의 파일로 이미지를 빌드하고 컨테이너를 띄웠습니다.

```
docker build -t db ex05/db                # ex05/db 폴더의 Dockerfile로 이미지 빌드
docker run -dit -p 3306:3306 db           # db 컨테이너 실행. 호스트 3306 → 컨테이너 3306
```



```

$ docker build -t db ex05/db
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "MYSQL_ROOT_PASSWORD") (line 7)
View build details: docker-desktop://dashboard/build/desktop-linux/s40kn0konmqxbyvoabl3c6up
$ docker run -dit -p 3306:3306 db
ad3d9b0d81fe86d6185bb1b865545d914f2be742c7046d5c6b1d984c62ff05
    
```

그림 3-27. MySQL 컨테이너 실행 로그

빌드 도중 환경 변수에 민감한 정보를 넣지 말라는 안내가 한 줄 났습니다. 위에서 본 `:::note` 블록의 그 이야기였습니다. 이번에는 학습용으로 그대로 두고, 컨테이너가 정상적으로 났는지 안에 들어가서 확인하기로 했습니다. `docker ps` 로 컨테이너 ID를 한번 본 다음, `docker exec` 로 내부에 들어갔습니다.

```

docker ps                # 실행 중인 컨테이너 확인
docker exec -it ad3d bash # MySQL 컨테이너 내부 접속
    
```



```

$ docker ps
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
ad3d9b0d81fe  db     "docker-entrypoint.s..."  10 seconds ago  Up 10 seconds  0.0.0.0:3306→3306/tcp, [::]:3306→3306/tcp  admiring_burnell
$ docker exec -it ad3d bash
bash-5.1#
    
```

그림 3-28. MySQL 컨테이너 내부 진입

컨테이너 안에서 MySQL 클라이언트를 띄웠습니다. 접속 정보는 Dockerfile에 적어 둔 값 그대로 넣었습니다.

```

mysql -u metacoding -p      # MySQL 접속. 비밀번호로 metacoding1234 입력
    
```

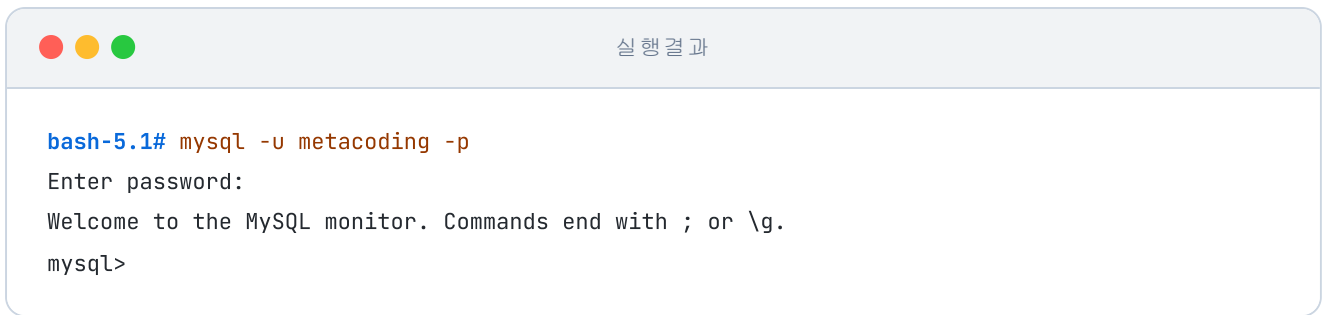


그림 3-29. MySQL 접속 성공

접속이 떨어진 자리에서 DB 목록과 테이블, 그 안에 들어 있을 초기 데이터를 차례대로 조회해 봤습니다.

<code>show databases;</code>	-- MySQL 서버에 있는 DB 목록 조회
<code>use metadb;</code>	-- metadb를 사용할 DB로 선택
<code>show tables;</code>	-- metadb 안의 테이블 목록 조회
<code>select * from user_tb;</code>	-- user_tb 테이블의 전체 행 조회

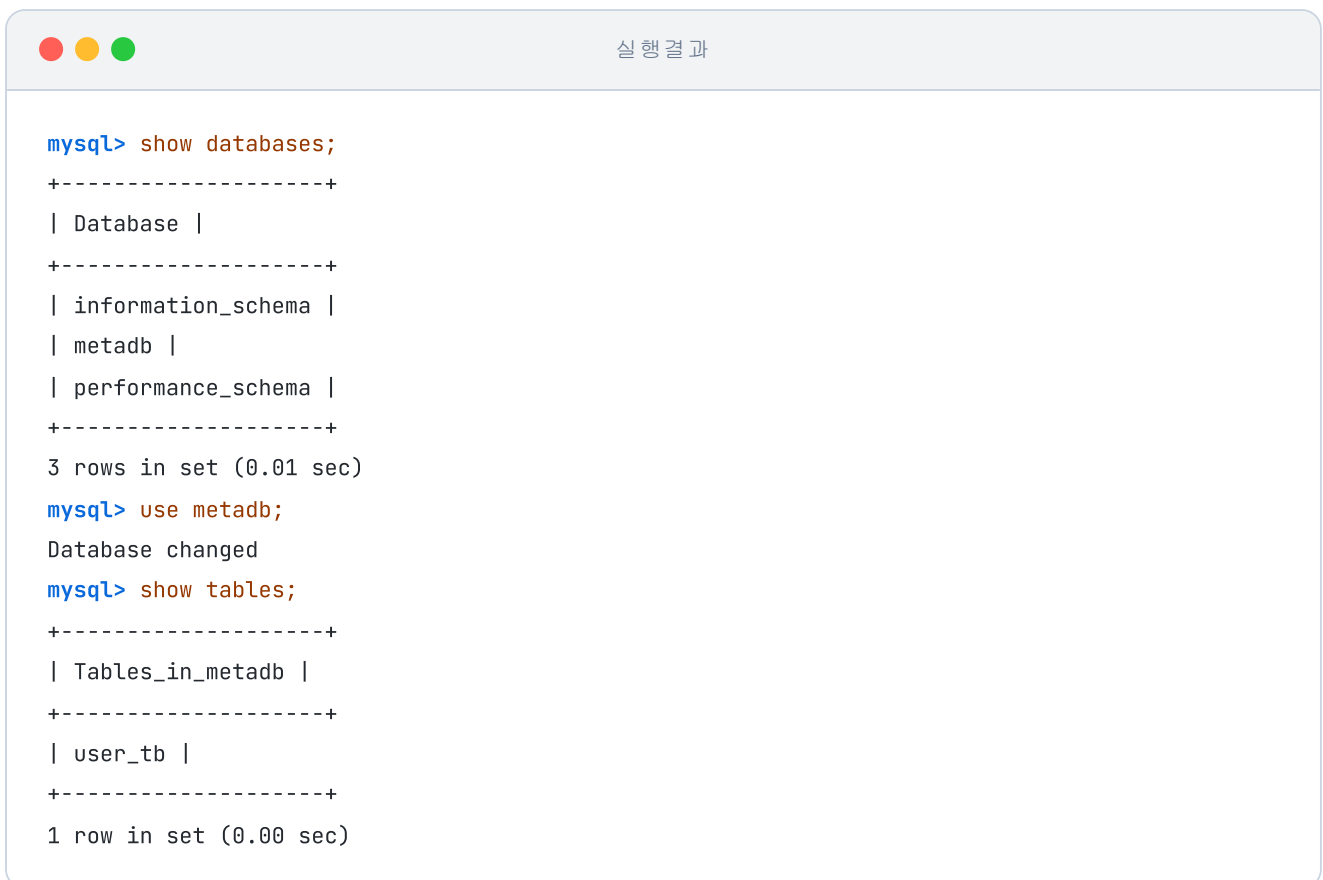


그림 3-30. 데이터베이스 목록 확인

실행 결과

```
mysql> select * from user_tb;
+----+-----+
| id | name |
+----+-----+
| 1  | ssar |
| 2  | cos  |
+----+-----+
2 rows in set (0.00 sec)
```

그림 3-31. user_tb 데이터 조회 결과

`init.sql`에 적어 둔 초기 데이터가 테이블 안에 그대로 들어와 있었습니다. 회의실에서 받은 통합 사이트의 회원 정보 자리가 이렇게 한 칸 채워졌습니다.

DB까지 컨테이너로 잡아 두고 나서 지난 며칠 동안 친 명령어를 돌아봤습니다. `docker build`, `docker run`, 네트워크 묶기, 포트 매핑이 같은 패턴으로 거듭 나타났습니다. 컨테이너가 한 대씩 늘어날 때마다 이 패턴이 한 줄씩 더 붙었습니다.

'프론트엔드, 백엔드, DB까지 셋이 같이 도는데 이걸 매번 한 줄씩 친다고? 한 번에 묶어 줄 방법을 찾아보자.'

흩어져 있는 컨테이너를 한 자리에 묶어 줄 도구를 찾아보기 시작했습니다.

3.5 Docker Compose - 여러 컨테이너를 한 번에

3.5.1 docker run 반복의 한계

수동 세팅이 피곤해서 Dockerfile을 손에 익혔습니다. 그런데 그 Dockerfile로 만든 이미지를 한 줄씩 띄우다 보니 결국 또 다른 반복 작업에 갇혀 있었습니다.

'프론트엔드와 백엔드, DB까지 한 번에 묶어서 한 줄로 띄울 방법이 있어야 하는데.'

기존 방식 — 컨테이너마다 build-run을 반복

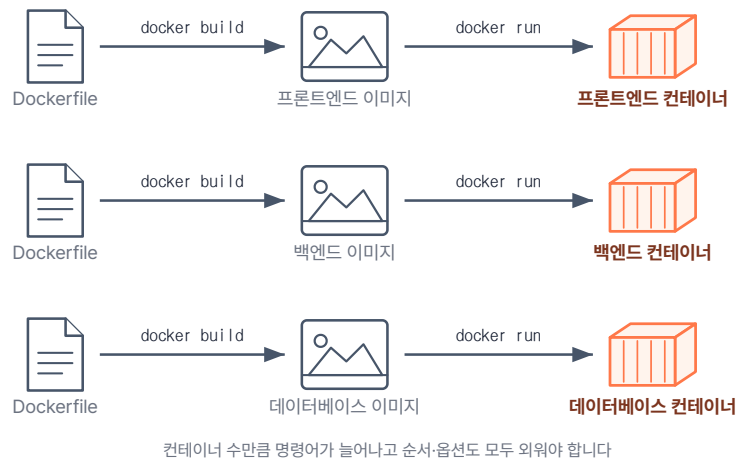


그림 3-32. 기존 방식 — 개별 빌드 및 실행 반복

검색 끝에 마주친 도구가 Docker Compose였습니다. 컨테이너마다 따로 명령을 치는 대신, 한 파일에 모든 컨테이너 정의를 적어 두고 명령 한 줄로 동시에 띄울 수 있는 도구였습니다.

Docker Compose 방식 — 한 줄 명령으로 여러 컨테이너 동시 실행

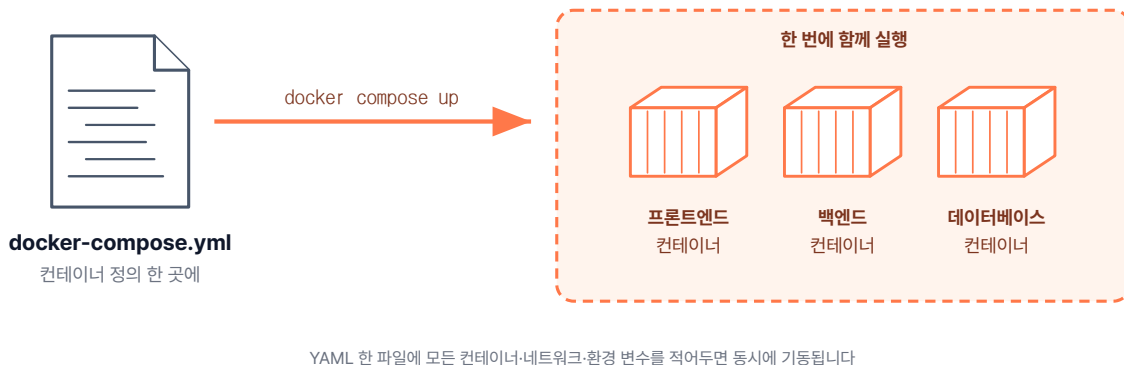


그림 3-33. Docker Compose 방식 — 한 번에 생성 및 연결

Compose가 받쳐 주는 자리는 셋이었습니다.

- **순서:** 어떤 컨테이너(예: DB)가 먼저 시작되어야 하는지 기동 순서를 지정할 수 있습니다.
- **네트워크:** 같은 Compose 파일에 정의된 컨테이너들은 자동으로 하나의 네트워크에 묶입니다. 이제 **docker network create** 를 따로 할 필요가 없습니다.
- **일괄 관리:** 명령어 한 줄로 모든 서비스를 시작하고, 한 줄로 종료할 수 있습니다.

Docker Compose: 여러 컨테이너를 하나의 YAML 파일(.yml)에 묶어 관리하는 도구입니다. 복잡한 컨테이너 간의 연결 고리와 실행 옵션을 문서화해두고, 이를 통째로 실행하거나 중지할 수 있게 도와줍니다.

3.5.2 docker-compose.yml 기본 구조

설정 파일의 뼈대는 생각보다 단순했습니다. 자주 쓰이는 옵션을 한 자리에 모아 모양만 익혔습니다.

```
services:
  <서비스명>: # 예: app, db, proxy 등
    container_name: <컨테이너명> # 실제로 생성될 컨테이너 이름
    image: <이미지명> # Docker Hub에서 이미지를 가져와야 한다면 여기 이미지명 작성
    build: <경로> # Dockerfile로 직접 이미지를 빌드한다면 여기 경로 입력
    ports:
      - "호스트포트:컨테이너포트"
    depends_on:
      - <먼저 해야 할 서비스>
    environment:
      - KEY=VALUE # 환경 변수 설정
    volumes:
      - <호스트경로:컨테이너경로> # 데이터 보관을 위한 볼륨 연결
    networks:
      - <네트워크명> # 아래 networks에서 만든 망에 이 컨테이너를 참여시킴

networks:
  <네트워크명>: # 이 프로젝트에서 사용할 네트워크 이름을 생성
```

3.5.3 실습: Compose로 ex01 다시 만들기

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex06>

연습용으로는 앞에서 손에 익힌 ex01을 그대로 다시 짜는 게 가장 빨라 보였습니다. Dockerfile은 그대로 두고, 세 컨테이너를 묶을 `docker-compose.yml` 한 장만 새로 적었습니다.

ex06/docker-compose.yml

```
services: # 컨테이너 단위로 실행할 서비스 묶음
  app1: # 첫 번째 서비스. 다른 서비스가 호스트명으로 호출
    build:
      context: ./app1 # ./app1 폴더의 Dockerfile로 이미지 빌드
    networks:
      - ex06-network # ex06-network에 연결
  app2: # 두 번째 서비스
    build:
      context: ./app2
    networks:
      - ex06-network
  lb: # 로드밸런서 서비스
    build:
      context: ./lb
    ports:
      - 80:80 # 호스트 80 → 컨테이너 80. 외부 진입점
    networks:
      - ex06-network

networks:
  ex06-network: # 세 서비스가 공유하는 사용자 정의 네트워크
```

Compose는 네트워크도 자기가 알아서 만들어 줍니다. 그래서 `lb/nginx.conf` 에서 백엔드 주소를 `host.docker.internal:포트` 대신 `app1:80`, `app2:80` 처럼 서비스 이름으로 그대로 적을 수 있게 됐습니다.

터미널을 ex06 폴더로 옮기고 한 줄을 쳤습니다.

```
cd ex06
docker compose up # 현재 폴더의 docker-compose.yml 기준으로 일괄 실행
```



실행 결과

```
$ docker compose up
[+] Running 4/4
✓ Network ex06_ex06-network Created
✓ Container ex06-app1-1 Created
✓ Container ex06-app2-1 Created
✓ Container ex06-lb-1 Created
Attaching to app1-1, app2-1, lb-1
```

그림 3-34. docker compose up 실행 결과

빌드부터 실행까지 한 줄에 다 들어갔습니다. 브라우저에서 본 결과 화면은 ex01 때와 같았는데, 그 자리까지 가는 명령어가 한 줄로 줄어 있었습니다.

'한 화면을 띄우는 데 명령어를 다섯 줄, 여섯 줄씩 나열하던 게 결국 이 한 줄로 끝나네.'

3.6 종합 실습 - 통합 웹사이트

전체 실습 코드는 깃허브를 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex07>

여기까지 컨테이너 한 대씩 띄우는 법과 Compose로 묶는 법을 따로따로 손에 익혔습니다. 이제 지금까지 익힌 것을 모아 프론트엔드·백엔드·DB 세 컨테이너로 구성된 통합 사이트를 만들어 봅니다. `docker compose up` 한 줄로 셋이 같이 올라가는 게 목표입니다.

3.6.1 전체 아키텍처



그림 3-35. 세 컨테이너로 구성되는 웹 애플리케이션 아키텍처

세 컨테이너의 역할은 다음과 같습니다.

컨테이너	역할
NGINX (frontend)	화면(index.html) 응답 + <code>/api/</code> 요청을 backend 컨테이너로 프록시
Spring (backend)	<code>/api/users</code> 요청을 받아 MySQL에서 사용자를 꺼내 JSON 반환
MySQL (db)	회원 데이터 보관 (3.4와 동일 구성)

ex07 폴더를 열었습니다. 백엔드, DB, 프론트엔드가 각자 하위 폴더에 들어 있고, 그 위에 `docker-compose.yml` 이 한 장 얹혀 있는 모양이었습니다.

```
ex07/
├── backend/           # Spring Boot 백엔드
│   ├── Dockerfile    # JDK 이미지 + entrypoint.sh 복사
│   └── entrypoint.sh  # Git clone + Gradle 빌드 + JAR 실행
├── db/               # MySQL. ex05와 동일한 구성
│   ├── Dockerfile    # MySQL 이미지 + init.sql 복사
│   └── init.sql       # 테이블·초기 데이터 생성 스크립트
├── frontend/         # NGINX + HTML
│   ├── Dockerfile    # nginx 이미지 + index.html·nginx.conf 복사
│   ├── index.html     # 로그인/게시판 UI
│   └── nginx.conf     # /api 경로를 backend로 프록시
├── docker-compose.yml # 세 서비스 + 네트워크 정의
└── README.md         # 실습 안내
```

3.6.2 Backend - 시작 시 소스 내려받아 빌드

자바 백엔드 한 장 정리

- **Spring Boot:** 자바로 만든 백엔드 프레임워크. API 서버를 빠르게 띄울 때 자주 쓰입니다.
- **JDK(Java Development Kit):** 자바 코드를 컴파일·실행하는 환경입니다.
- **Gradle:** 자바 빌드 도구. 소스 코드를 묶어 실행 파일로 만들어 줍니다.
- **JAR:** 자바 실행 파일 묶음. `java -jar app.jar` 로 실행합니다.

backend 폴더를 펼치자 한 가지 자리가 눈에 들어왔습니다. Spring Boot 서버라고 적혀 있는데도 폴더 안에 자바 소스가 한 장도 없었습니다. 그 자리에 `entrypoint.sh` 라는 셸 스크립트가 한 장 들어 있었습니다.

이 스크립트는 컨테이너가 뜨는 그 순간 깃허브에서 소스를 받아 와서, 그 자리에서 빌드하고 실행하도록 짜여 있었습니다. 한 번 적어 두면 누가 어디서 컨테이너를 띄워도 같은 모양의 백엔드가 떨어집니다.

파일	역할
Dockerfile	JDK 21 이미지 + git 설치 + entrypoint.sh 실행 지정
entrypoint.sh	컨테이너 시작 시 깃허브에서 소스 받기 + Gradle 빌드 + JAR 실행

Dockerfile vs entrypoint.sh

- **Dockerfile**은 이미지 **빌드** 단계입니다. 컴퓨터를 조립해 셋업하듯, 한 번만 실행되며 이미지 안에 어떤 파일·도구를 넣을지 정의합니다.
- **entrypoint.sh**는 컨테이너 **시작** 단계입니다. 컴퓨터를 켤 때마다 자동 실행되는 시작 프로그램처럼, 컨테이너가 뜰 때마다 실행되며 소스 다운로드·빌드·서버 기동처럼 켜지자마자 해야 할 일을 적어둡니다.

ex07/backend/entrypoint.sh

```
#!/bin/bash
git clone https://github.com/metacoding-10-linux-docker/backend-server # 백엔드 소스
내려받기
cd backend-server # 클론한 폴더로 이동
chmod +x gradlew # 실행 권한 부여
./gradlew build # Gradle로 JAR 빌드
java -jar -Dspring.profiles.active=prod build/libs/*.jar # prod 프로파일로 실행
```

이 스크립트가 컨테이너가 뜰 때 자동으로 도는 자리에 들어가도록, Dockerfile에서 ENTRYPOINT로 지정해 뒀습니다.

ex07/backend/Dockerfile

```
FROM eclipse-temurin:21-jdk # JDK 21 공식 이미지 사용
WORKDIR /var/current/app # 작업 디렉토리 설정
COPY entrypoint.sh /entrypoint.sh # 위의 스크립트 복사
RUN apt-get update && apt-get install -y git # git clone에 필요한 git 설치
ENTRYPOINT ["/entrypoint.sh"] # 컨테이너 시작 시 스크립트 실행
```

올라온 백엔드 서버 자체는 단순했습니다. `/api/users` 요청이 오면 DB에서 회원 목록을 꺼내 JSON으로 돌려주는 컨트롤러 한 개였습니다.

'이미지 안에 소스 가져오는 절차를 같이 묶어 두니, 환경에 상관없이 코드 한 곳만 잡혀 있으면 같은 백엔드가 그대로 뜨네.'

3.6.3 DB - 3.4와 동일한 MySQL 컨테이너

DB 자리에는 3.4에서 만진 MySQL 구성을 그대로 가져왔습니다. 이미지도 `init.sql` 도 손댈 데가 없었습니다.

파일	역할
Dockerfile	MySQL 이미지 + 환경 변수(계정·비밀번호·DB명) + init.sql 복사
init.sql	컨테이너 첫 기동 시 자동 실행되어 user_tb 테이블과 초기 데이터 생성

3.6.4 Frontend - 정적 페이지 + API 프록시

frontend 폴더에는 두 장이 들어 있었습니다.

파일	역할
index.html	jQuery로 <code>/api/users</code> 를 호출해 받은 사용자 목록을 화면에 그림
nginx.conf	정적 페이지(<code>/</code>)는 index.html로, API 요청(<code>/api/</code>)은 backend 컨테이너로 라우팅

ex07/frontend/nginx.conf


```
events {}

http {
    upstream backend {
        server backend:8080;           # backend = docker-compose.yml의 서비스 이름
    }

    server {
        listen 80;
        root /usr/share/nginx/html;   # 정적 파일 위치

        location / {                  # 슬래시 요청
            index index.html;         # 기본 HTML 응답
        }

        location /api/ {              # /api 로 시작하는 요청
            proxy_pass http://backend; # 위 upstream으로 프록시
        }
    }
}
```

핵심은 백엔드 주소를 IP가 아니라 `backend:8080` 처럼 서비스 이름으로 적었다는 점이었습니다. Compose가 자동으로 만들어 준 네트워크에서 서비스 이름이 그대로 호스트명으로 통합니다.

3.6.5 docker-compose.yml

마지막으로 셋을 한 자리에 모으는 설정 파일을 적었습니다.

ex07/docker-compose.yml

```

services:
  backend: # Spring Boot 백엔드 서비스
    build:
      context: ./backend # ./backend 폴더의 Dockerfile로 빌드
    environment: # 컨테이너에 주입할 환경 변수
      # DB 접속 URL (호스트명 db = 아래 db 서비스명)
      # 책 표기를 위한 줄바꿈 (실제는 한 줄)
      SPRING_DATASOURCE_URL: "jdbc:mysql://db:3306/metadb\
        ?useSSL=false\
        &serverTimezone=UTC\
        &allowPublicKeyRetrieval=true"
      SPRING_DATASOURCE_USERNAME: metacoding # DB 계정
      SPRING_DATASOURCE_PASSWORD: metacoding1234 # DB 비밀번호
    networks:
      - ex07-network # 공용 네트워크 연결

  db: # MySQL DB 서비스
    build:
      context: ./db
    networks:
      - ex07-network

  frontend: # nginx 프론트엔드 서비스
    build:
      context: ./frontend
    ports:
      - "80:80" # 호스트 80 → 컨테이너 80. 외부 진입점
    networks:
      - ex07-network

networks:
  ex07-network: # backend·db·frontend가 공유하는 네트워크

```

ex07 폴더로 들어가 한 줄을 입력했습니다.

```

cd ex07
docker compose up # 현재 폴더의 docker-compose.yml 기준으로 일괄 실행

```

실행 결과

```

$ docker compose up
#1 [internal] load local bake definitions
#1 reading from stdin 1.78kB 0.0s done
#1 DONE 0.0s
#2 [backend internal] load build definition from Dockerfile
#2 transferring dockerfile: 214B 0.0s done
#2 DONE 0.0s
#3 [frontend internal] load build definition from Dockerfile
#3 transferring dockerfile: 172B 0.0s done
#3 DONE 0.0s
#4 [db internal] load build definition from Dockerfile
#4 transferring dockerfile: 302B 0.0s done
#4 DONE 0.0s
#5 [frontend internal] load metadata for docker.io/library/nginx:latest
#5 DONE 0.1s
#6 [db internal] load metadata for docker.io/library/mysql:latest
#6 ...
[+] Building 30.5s (15/15) FINISHED
[+] Running 4/4
✓ Network ex07_ex07-network Created
✓ Container ex07-db-1 Started
✓ Container ex07-backend-1 Started
✓ Container ex07-frontend-1 Started

```

그림 3-36. docker compose up 한 줄로 세 컨테이너가 동시에 뜨는 모습

엔터 한 번에 빌드 로그가 위로 줄줄이 올라갔습니다. 백엔드 자리는 첫 실행이라 깃허브에서 소스를 받고 Gradle이 한 번 도는 시간이 더 들어갔습니다. 빌드가 끝나길 기다렸다가 브라우저에 `localhost:80`을 입력했습니다.

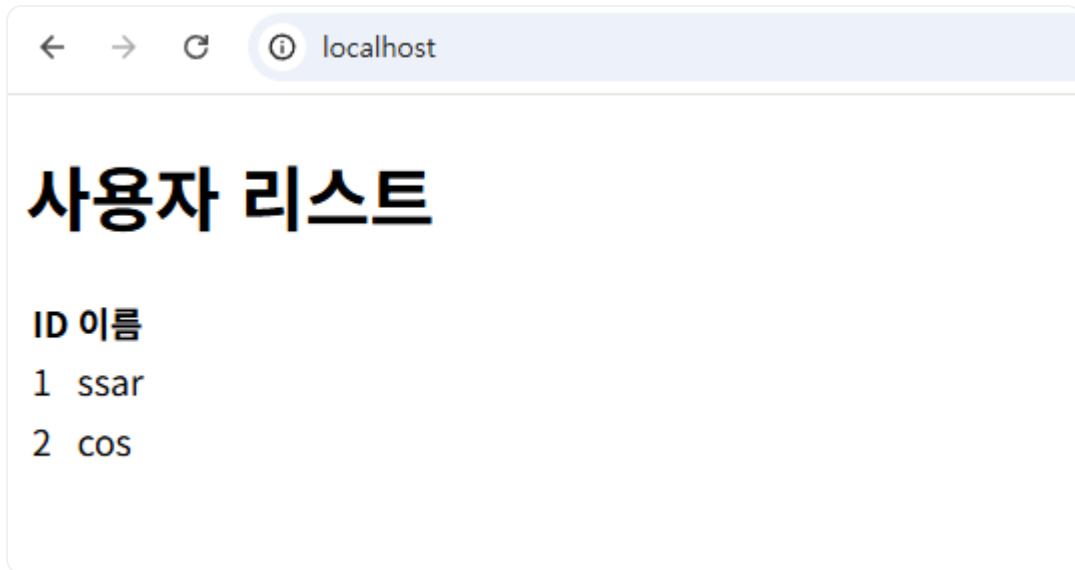


그림 3-37. 사용자 목록 조회 성공

화면에 회원 목록이 떴습니다. NGINX가 받아 백엔드로 넘기고, 백엔드가 DB에서 꺼낸 데이터가 그 자리에 그대로 올라온 결과였습니다.

다음 날 회의 시간이 다가왔습니다. 회의실 가운데 자리에 노트북을 펼치고 빔 프로젝터에 연결했습니다. 팀장과 동료가 자리에 앉았습니다.

오픈이 : "한 줄만 칠게요. 셋이 같이 올라오는지 보시면 돼요."

엔터를 눌렀습니다. 빔에 컨테이너 세 개의 빌드 로그가 줄줄이 올라갔습니다. 마지막 줄이 떨어지자 브라우저에 회원 목록이 떴습니다. 화면 안에 화이트보드의 네모 세 개가 그대로 들어와 있었습니다.

팀장 : "환경 구성은 깔끔하게 됐네요. 잘했어요."

회의실을 나오는 길에 어깨가 한결 가벼웠습니다. 며칠 전 같은 자리에서 받은 프로젝트가 명령 한 줄로 마무리됐습니다.

이것만은 기억하자

- **Dockerfile은 환경 구성의 레시피입니다.** 베이스 이미지·설치할 패키지·실행 명령을 적어두면 어디서든 동일한 환경을 자동으로 재현합니다.
- **NGINX는 서버 앞단의 리버스 프록시입니다.** 경로 라우팅·로드밸런싱·캐싱이 핵심이며, 설정 파일의 뼈대는 늘 비슷합니다.

- **Redis는 서버들이 함께 들여다보는 화이트보드입니다.** 서버가 여러 대로 늘어도 세션을 외부 저장소에 두면 로그인 상태가 흩어지지 않습니다.
- **사용자 정의 네트워크는 컨테이너끼리 이름으로 통신하게 해줍니다.** `host.docker.internal` 처럼 호스트를 우회할 필요가 없어집니다.
- **Docker Compose는 여러 컨테이너를 한 파일로 관리하는 설계도입니다.** 명령어 한 줄로 빌드·네트워크·의존 관계가 자동으로 묶입니다.

자리로 돌아와 의자에 앉으니 모니터에 아직 회원 목록 화면이 떠 있었습니다. 시연이 끝났다는 안도감이 가라앉은 자리에 다른 생각이 차고 들어왔습니다. 지금까지 도커로 한 일은 결국 **띄우기**에 가까웠습니다. 한 줄로 같이 띄울 수는 있게 됐지만, 자리를 비운 사이에 무슨 일이 벌어지면 그 다음을 받쳐 줄 도구는 비어 있었습니다.

'지금은 내가 수동으로 챙기지만, 새벽 두 시에 컨테이너 하나가 죽으면 누가 다시 살려 줄까.'

자동 복구와 무중단 배포, 다중 서버 관리, 설정 분리까지 운영 자리에서 마주칠 이 숙제들을 한 묶음으로 받쳐 주는 도구가 다음 챕터의 주인공 **쿠버네티스(Kubernetes)** 입니다.